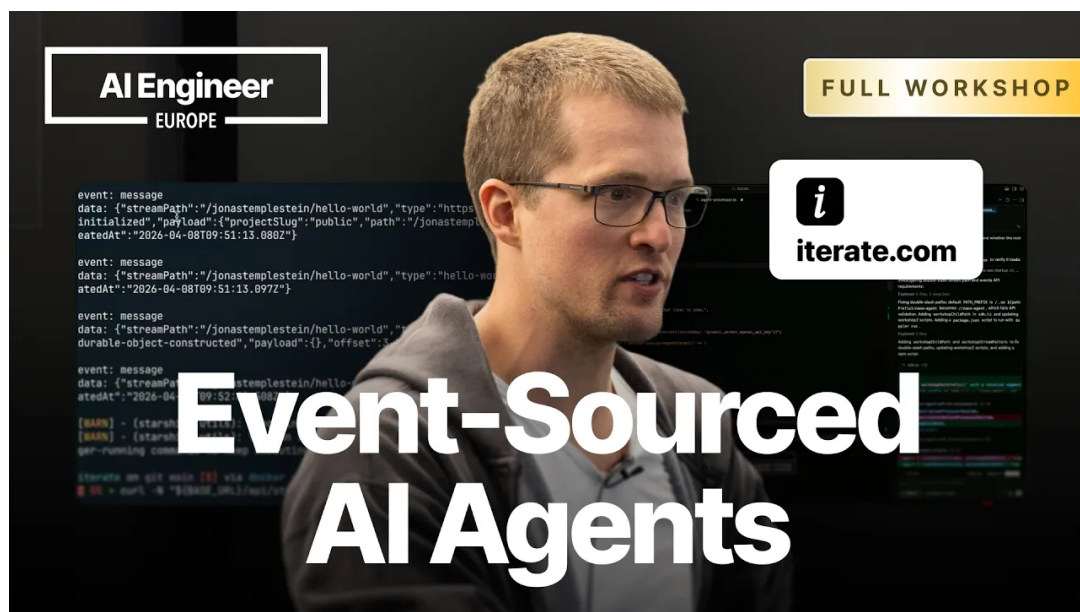


课程笔记

用流处理器打造事件溯源 AI Agent：架构设计实战解析

Jonas Templestein, Misha & Codex

2026 年 5 月 16 日



视频频道：AI Engineer

发布日期：2026 年 5 月 14 日

视频时长：01:04:26

视频链接：<https://www.youtube.com/watch?v=vi-2nasppAg>

目录

1	为什么要把 Agent Harness 做成事件溯源系统	4
1.1	Agent harness 的真实痛点：副作用逃出日志	4
1.2	可扩展和可组合：把实验面收缩到事件	4
1.3	公开 HTTP URL：让 agent 真正接入互联网	5
1.4	分布式插件：能力放大，也放大竞态和循环	6
1.5	本章小结	7
2	事件流的最小原语：path、event envelope、offset 与订阅	7
2.1	path 即流：像文件路径一样定位 agent	7
2.2	event envelope：业务事实外面的统一信封	8
2.3	SSE/live stream：用 HTTP 长连接观察新事件	10
2.4	错误、幂等、暂停与熔断也事件化	11
2.5	schedule、heartbeat 与 push/pull subscription	11
2.6	本章小结	12
3	一种 API 视角：所有 Agent 行为都落成事件	12
3.1	问题：插件 API 为什么会越长越难用	12
3.2	机制：streaming chunk、tool call、error、user input 都可事件化	14
3.3	定义：programmable stream 是什么	15
3.4	边界：TypeScript 是 workshop 选择，目标是 polyglot	15
3.5	本章小结	16
4	从 curl 到 TypeScript：stream processor 的 reducer/after-append 分工	16
4.1	从 SDK 调用看见同一条事件流	16
4.2	正式定义：stream processor、reducer 与 after-append	18
4.3	ping-pong processor：最小可工作的分工	19
4.4	processor runtime：把事件流接到处理器	20
4.5	重启 catch-up：旧事件可以重算，旧副作用不能乱放	21
4.6	本章小结	21
5	把 LLM Agent 状态化：输入事件、历史、系统提示与响应流	22
5.1	子路径与 sub-agent：把路径当作协作空间	22

5.2	Agent state: model、system prompt、history 与运行 flag	23
5.3	输入事件: agent input added 极简到只有字符串	24
5.4	Responses API role/content 映射: 从自家事件到模型消息	25
5.5	after-append: 用状态发起 LLM 请求	26
5.6	响应事件与 history 归约: 流式输出也要落账	27
5.7	现场边界: 思想成立, 代码不稳定	28
5.8	本章小结	28
6	失败的现场 demo 反而暴露了真实架构: 内置 processor、UI 投影与 reducer 复用	29
6.1	失败现场给出的有效信息	29
6.2	内置 circuit breaker processor: 用事件保护事件流	30
6.3	正式定义: projection 与 read model	31
6.4	UI 也是 processor: 读侧同样在 reducing	32
6.5	reducer 可导入复用: 共享事件类型上的抽象叠加	33
6.6	失败 demo 的工程含义	34
6.7	本章小结	34
7	动态 Worker: 把处理器代码打包成事件	34
7.1	dynamic worker 的正式定义	34
7.2	stream 隐式创建: 先有路径, 再长出行为	35
7.3	configured payload: script 字符串里的 reducer 与 after-append	37
7.4	AI agent 版本: processor file bundle 成 fat event	38
7.5	secret 不能进 event: env 与 secret 注入	38
7.6	npm 依赖需要 bundle: 输入框适合小脚本, 工程代码需要打包	39
7.7	错误也事件化: 编译失败、运行失败、修复建议	39
7.8	三种运行路径: 本地 SSE、Web service、Cloudflare dynamic worker	40
7.9	本章小结	40
8	总结与延伸: 分布式插件、安全检查、最终一致性与权限边界	41
8.1	第三方安全 checker 的 200ms 窗口	41
8.2	为什么第三方 before hook 风险很高	43
8.3	eventual consistency: 赶上就补, 赶不上继续	43
8.4	durable stream: append-only event log 加 offset tracking	44

8.5	Kafka 与 PubSub: offset 谁来跟踪	45
8.6	public namespace 的认证、授权与 client provenance	46
8.7	全文收束: 一个事件流, 多个 processor, 可审计副作用	46
8.8	本章小结	47

1 为什么要把 Agent Harness 做成事件溯源系统

Jonas 在开场的判断很直接：他想构建 agent harness 的方式，是把它做成 purely event sourced 的系统。这里先定义两个本章会反复使用的词。

Agent harness (Agent 外壳) 指包住 LLM 的运行框架。它负责把用户输入、系统提示、历史消息、工具调用、流式输出、错误和外部 API 调用组织成一个可运行的 agent。可以把 LLM 想成发动机，harness 就是油路、仪表盘、刹车、传动和车载记录仪。发动机本身只产生推力；harness 决定输入从哪里来，动作怎样执行，失败时能不能查清现场。

Event Sourcing (事件溯源) 指把系统中已经发生的事实写进只追加的事件日志，再从日志重建当前状态。它像会计账本：余额并非单独凭空存在，而是由一笔笔流水累加得出。若某条事件流记作

$$E = \langle e_1, e_2, \dots, e_n \rangle,$$

那么状态可以被理解为从这些事实一步步推导出来的结果。严格的归约公式、事件信封、offset 和处理器分工会在后续章节展开；本章只使用这个直觉：**事实先落日志，状态和动作再围绕日志展开。**

1.1 Agent harness 的真实痛点：副作用逃出日志

许多 agent 系统表面上已经有日志。聊天历史是一种日志，工具调用记录是一种日志，LLM streaming chunk 也常常被某种 trace 系统捕获。问题出在副作用经常逃到这些日志之外：某个工具请求已经发出，某封 Slack 消息已经推送，某次 HTTP webhook 已经命中第三方服务，后来只能去 traces、控制台输出或云平台请求记录里拼现场。

这会让调试变成考古。开发者看到 agent 最终给出一个奇怪答案，却无法沿着同一条事实链追问：它什么时候收到输入？什么时候决定调用工具？工具返回了什么？外部系统有没有被写入？哪个步骤触发了下一轮 LLM 请求？如果这些动作分散在日志、trace、数据库行、队列消息和供应商控制台里，单次事故就会变成多系统对表。

事件溯源对 agent harness 的第一价值是 debugable。它要求“可能影响 agent 行为的事实”尽量进入同一条可回放、可检查的事件流。这样排查问题时看的对象从一堆散落的副作用痕迹，收敛为一条按发生顺序排列的事实账本。

这个约束还有一个边界：并非所有外部世界都能被塞回日志。真实 HTTP 请求已经发出，真实邮件已经发送，真实支付已经扣款，事件流只能记录“发出过什么”和“收到过什么”。所以关键是让副作用的输入、输出、错误与触发原因都留下事件证据，避免只剩一行无法复现的 trace。

1.2 可扩展和可组合：把实验面收缩到事件

Jonas 提到 extensible 时，重点并不限于“人写插件”。更有意思的是 agent 也能扩展自己：当新的能力以事件方式接入，agent 可以观察、组合、甚至生成新的处理逻辑。这里的工程收益来

自接口面的收缩。插件不必理解整个 harness 的内部对象模型，只需要围绕事件工作：看见某类事实，追加另一类事实，或者把外部系统的响应写回流里。

composable 的意义也在这里。假设一个插件负责把 web form 提交转换成 agent 输入，另一个插件负责把安全检查结果写入上下文，第三个插件负责把最终回答推送到 Slack。传统 harness 往往需要为每个插件规定专门的生命周期钩子、上下文对象和回调顺序。事件流方案把组合方式换成“多个参与者围绕同一事实账本协作”：每个插件只承诺自己会读哪些事件、写哪些事件。

“可组合”在这里类似 Unix pipe 的精神：每个程序做一小段清晰转换，组合后得到更复杂的行为。区别是 agent 的管道并非只传一串字节，它传的是按时间排列的事实。某个插件写入的事件会成为另一个插件的输入，组合关系因此显式留在日志里。

这种组合带来试验速度。讲者认为当下还没有人完全知道 agent harness 的最佳配方；若所有实验都必须在单进程、单线程、单台机器上改核心代码，试错成本会很高。事件流把实验边界往外推：插件可以单独迭代，失败时留下事件，成功时成为另一个处理器可以依赖的事实来源。

1.3 公开 HTTP URL：让 agent 真正接入互联网

Section 1 里最有冲击力的主张，是“agent 一旦存在，就应该有一个 URL”。这里的 publicly routable 指 agent 暴露一个可被互联网访问的 HTTP 入口。Web form 提交、Slack webhook、第三方系统通知，都可以直接变成事件流里的输入。

机制很朴素：HTTP 请求命中某个路径，服务把请求内容转成事件并追加到对应 agent 的流里。用户填完网页表单，表单结果就是 agent 的新输入；Slack 发来 webhook，webhook payload 就进入同一条事实链；第三方系统不需要安装一个特殊 connector，也不需要理解 harness 内部结构，只要会发 HTTP。

这解释了 Jonas 对“connector 概念”的不耐烦。很多系统先造出一个封闭 agent，再倒过来为 Slack、表单、CRM、CI、issue tracker 分别写连接器。公开 URL 把方向调直：外部世界天然会说 HTTP，agent harness 接收 HTTP 后把它登记为事件。这样入口统一，审计也统一。

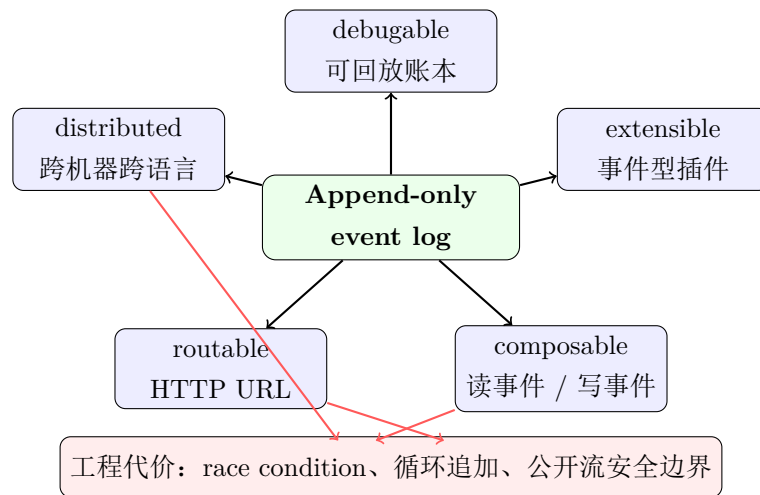


图 1: 事件溯源 Agent Harness 五个架构目标

1

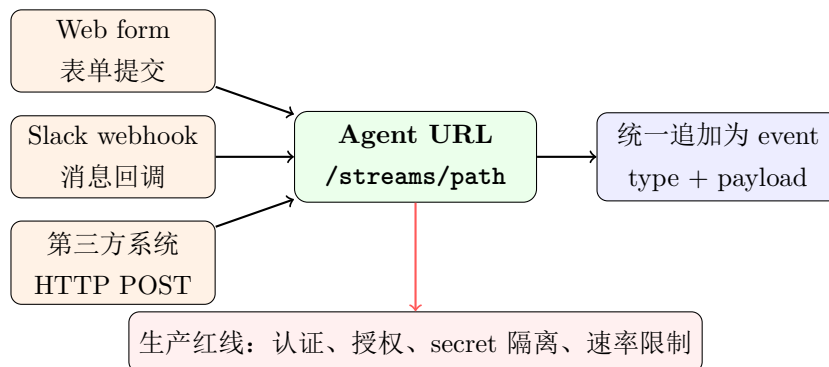


图 2: 公开 HTTP URL 把外部输入直接接入事件流

2

公开入口的边界必须说清楚。视频里的 workshop 服务没有认证，参与者能看到彼此的 event stream，payload 里也不能放 secret。这个设计用于演示很方便，用于生产会直接暴露数据、权限和滥用风险。认证、授权、client provenance 等内容会在最后一章详细收束；本章只提前标记红线：公开路由能力必须配套身份、权限、速率和审计。

1.4 分布式插件：能力放大，也放大竞态和循环

distributed 的目标是让 agent、插件和处理器可以运行在不同机器、不同进程、不同语言里。一个 agent 在服务器上运行，安全插件在另一个供应商的机器上运行，用户自己的插件用 Rust 写，团队内部插件用 TypeScript 写。只要它们围绕事件流交换事实，语言和部署位置就不再是核心障碍。

这带来的能力很现实。安全检查、RAG、监控、通知、审计、业务审批都可以作为独立处理器接入。某个团队不需要把自己的服务嵌进 agent 主进程，只需要订阅相关事件，再把结果追加

¹视频画面时间区间：00:01:52–00:05:01。

²视频画面时间区间：00:03:30–00:04:33。

回来。对商业插件来说，这也提供了一种接口：服务商观察某类 agent 行为，在有限时间内补充判断或上下文。

分布式插件是一把双刃剑。只要两个插件都能观察事件并追加新事件，就可能出现 race condition：A 插件和 B 插件基于同一旧状态同时写入互相冲突的结果。也可能出现无限循环：A 看见事件后写 B，B 看见 B 后写 A，最后事件流被 ping-pong 式追加撑爆。

race condition 的风险来自“同时看到不完整世界”。例如安全插件还没写入风险判断，执行插件已经根据旧上下文发出了外部请求；或者两个插件都认为自己应该处理同一个用户输入，于是各自追加了一条互斥的动作事件。事件溯源能保留完整事故现场，却不会自动消灭并发问题。后续章节会用暂停、熔断、幂等和处理器边界继续处理这些风险。

无限循环的风险更直观。插件 A 规定“看到待处理输入就补一条解释事件”，插件 B 规定“看到解释事件就补一条待处理输入”，二者组合后就会持续追加。真实系统需要速率限制和 circuit breaker；视频后面会展示超过一定事件速率时暂停流的机制。本章先抓住动机：当 harness 允许分布式扩展，循环保护就从附加功能变成生存条件。

1.5 本章小结

本章建立了 Jonas 方案的动机：agent harness 本来就围绕事件运行，关键是让影响行为的事实尽量进入同一条只追加日志。这样调试有账本，可扩展性来自统一事件接口，可组合性来自多个插件围绕同一事实源协作，公开 HTTP URL 让 web form、Slack webhook 和第三方系统可以直接进入 agent，分布式部署让插件跨机器、跨语言运行。

代价也很硬：公开流会引入认证和 secret 风险，分布式插件会引入 race condition，互相追加事件的插件会制造无限循环。后续章节会逐步补上事件流的最小原语、处理器分工、熔断、动态 worker 与最终一致性边界。

2 事件流的最小原语：path、event envelope、offset 与订阅

上一章把目标说清楚了：agent harness 要可调试、可扩展、可组合，还要能用普通 HTTP 接入外部世界。本章进入最小 API 层。Jonas 在 00:05:04 之后展示的 `events.iterate.com`，本质上先把 agent 压缩成一个非常朴素的对象：某个 path 下的一串 raw events。所有后续能力，包括 SSE 观察、webhook 去重、pause/resume、heartbeat、push subscription，都从这个最小对象长出来。

2.1 path 即流：像文件路径一样定位 agent

在这个服务里，**path** 是层级化的字符串，例如 `/jonas-templestein/hello-world`。它像文件系统路径：斜杠表达层级，前缀表达命名空间，完整路径定位一个具体资源。这个资源的内容是 append-only 的事件序列，读取它就像打开一份持续增长的流水账。

还有两个很容易在 API 层混淆的细节。第一，视频里的 path 以正斜杠开头，例如 `/jonas-templestein/hello-world`；这让它看起来像稳定路由。讲者也提到，服务端对斜杠的 URL encoding 很宽容，开发者既可以保留斜杠来获得更好的 curl 手感，也可以在客户端或代理要求时编码斜杠。第二，`project slug` 是 path 之外的命名空间开关。字幕里的说法接近“换一个 project slug 就像换一个全新的 database/namespace”，并且可以通过 HTTP header 传入。也就是说，`/workshop/alice/demo` 这样的 path 决定流的位置，project slug 决定它落在哪个隔离的项目空间里；生产设计不能把这两层权限边界混成一层。

path 的工程含义 这里的第一条规则是：**path 就是 event stream 的地址**。向某个 path 对应的 streams API 执行 append，相当于把一条事实写入那条流；从同一个 path 读取，相当于取回这条流已经发生过的事实。这个设计让 agent 很容易获得 URL：web form、Slack webhook、第三方服务、命令行 curl 都可以把外部输入落到同一条流里。

event stream（事件流）指同一路径下按照 offset 排列的事件序列，是 agent 的事实来源。可以形式化写成：

$$E = \langle e_1, e_2, \dots, e_n \rangle$$

其中 E 表示某个 path 上的 event stream， e_i 表示第 i 条事件，尖括号 $\langle \cdot \rangle$ 强调顺序。这里的顺序很关键：如果同一个用户先发 `hello`，再发 `pause`，再尝试发第二个 `hello`，系统行为取决于这些事件的先后位置。

这也解释了为什么 path 可以成为 agent harness 的最小数据库。传统数据库把状态直接存成“当前值”，事件流把事实写成“历史流水”。账本类比很直观：余额来自收入、支出、冲正一笔笔累加，单独看一个余额数字很难还原过程。事件流也是如此，agent 当前是否 paused、最近是否有 heartbeat、外部 webhook 是否已经处理，都可以从流里读出依据。

2.2 event envelope：业务事实外面的统一信封

Jonas 在 curl 里 append 一个 `hello world` 事件后，服务返回了一个基本形状：有 `type`，通常有 `payload`，服务端补上 `stream path`、`offset` 和 `created_at`。这就是 **event envelope（事件信封）**：包住业务 payload 的统一外壳，让不同来源、不同类型、不同处理器产生的事件都能用同一套读取方式消费。

如果已经用 path 选定了某条 event stream，一个理想化事件可以按题目要求简化写成：

$$e_i = (\tau_i, p_i, o_i, c_i)$$

其中：

- τ_i 是第 i 条事件的 `type`，例如 `hello.world` 或一个指向文档的 URL 形式类型；
- p_i 是 `payload`，承载业务数据，例如用户输入、webhook 原文、heartbeat 配置；

- o_i 是 `offset`，由服务端在写入时分配；
- c_i 是 `created_at`，记录服务端接受并创建事件的时间。

如果需要把事件从一条流搬到另一条流，或者把多个 `stream` 汇总到一个审计视图里，可以再显式带上 s_i 或 $path_i$ 表示 `path`。视频中的 `envelope` 实际返回了这个字段，因为同一个 UI 或 `processor` 可能同时观察多个 `path`。

offset（偏移量） 是事件在单条流内的单调递增序号，从 1 开始。它的机制含义是“服务端串行化点”：多个客户端同时向同一条 `stream` `append`，服务端必须决定谁先写入，随后给出 $o_1 = 1, o_2 = 2, \dots$ 这样的顺序。`offset` 像书页页码，也像 Kafka 里消费进度的坐标；它可以回答“处理到哪里了”“从哪里开始重放”“某个错误发生在第几条事实之后”。

`type` 为什么常写成 URL 视频里服务端生成的错误类型以 `https://events.iterate.com/...` 开头。它仍然只是一个 `opaque string`，并不要求所有 `type` 都是 URL。采用 URL 的好处是，字符串本身可以指向文档，读者看到事件类型时能顺手打开规范；采用短词的好处是 `curl` 和调试更轻便。边界在于：`type` 必须稳定，`payload schema` 可以演进，消费者需要知道自己能理解哪些 `type`。

客户端至少提交 `type`，多数事件还带 `payload`；服务端补齐流位置和时间坐标

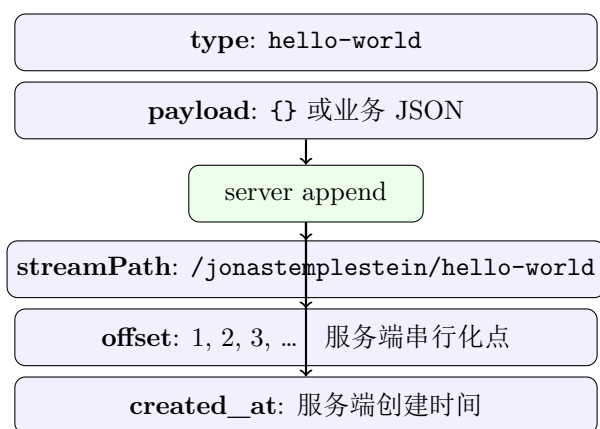
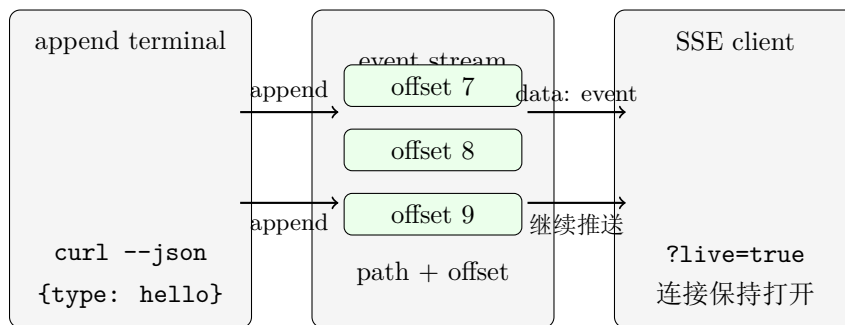


图 3: event envelope 把业务事实包进统一信封

3

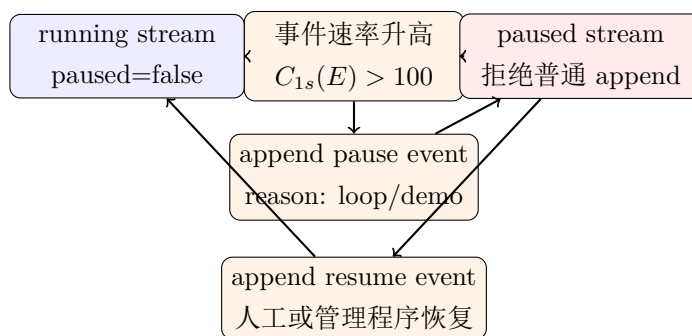
³视频画面时间区间：00:08:27–00:09:29；已抽取候选帧，因终端文字缩放后可读性不足，最终采用重绘结构图。



SSE 是单向 HTTP 长连接：读完历史后继续等待未来事件。断线恢复仍要依赖最后 offset。

图 4: SSE/live stream: append 后观察端持续收到新事件

4



熔断器的核心目标是阻断插件互相追加造成的无限循环，同时保留已经发生的事件现场。

图 5: pause 与 circuit breaker: 用事件化状态阻断无限循环

5

2.3 SSE/live stream: 用 HTTP 长连接观察新事件

读取事件流有两种直觉：一次性读取已有事件，或者保持连接等待未来事件。视频中 curl 加上 -N 后可以看到 streaming SSE；再加上 ?live=true，HTTP 连接会保持打开，后续 append 的事件会继续流入终端。

SSE (Server-Sent Events) 是一种基于 HTTP 长连接的单向事件推送机制。客户端发起请求，服务端持续写出 `data: ...` 这样的事件块。它适合这里的原因很具体：agent harness 的观察者通常只需要从服务端接收新增事件，暂时不需要在同一条连接上双向通信。浏览器 UI、命令行 curl、本地 processor 都可以用这条长连接观察同一个 stream。

⁴视频画面时间区间：00:09:32–00:10:31；已抽取候选帧，因双栏终端在版面内信息密度过高，最终采用重绘序列图。

⁵视频画面时间区间：00:12:08–00:13:42。

?live=true 的边界 ?live=true 的含义是“读完历史后继续等待未来事件”。它不会把 HTTP 变成消息队列的完整替代品，也不会保证客户端永远在线。真实系统仍然要记录最后看到的 offset；连接断开后，客户端用这个 offset 恢复读取，避免漏掉断线期间追加的事件。

一个极小 agent 已经可以由两端组成：一端 append 用户输入，另一端通过 SSE 观察输入并 append 响应。此时“智能”可能还来自人手、脚本或未来的 LLM processor，但外部接口已经成立：所有互动都通过事件流表达。

2.4 错误、幂等、暂停与熔断也事件化

服务对输入很宽容，原因是它面向 webhook、第三方系统和人手 curl。宽容并不等于没有约束。事件至少要有 type；如果 append 过来的 JSON 没有 type，服务不会把这条业务事件当成合法事实，而是追加一个类似 invalid event appended 的错误事件。

invalid event appended 的价值在于保留现场：错误请求没有静默丢失，后续调试者可以看到“某个客户端在某个时间试图写入无效事件”。不过视频也给了一个重要边界：当 stream 已经 paused 时，再 append 普通事件会被拒绝；这个拒绝不会继续写成无限错误事件，因为那会在死循环里制造更多事件。

idempotency key（幂等键）是表示“同一外部请求”的去重键。webhook 常见失败模式是：第三方服务发来请求，网络超时，第三方看不到成功响应，于是重试。没有 idempotency key 时，同一订单、同一 Slack 消息、同一表单提交可能被 append 两次；有了同一个 idempotency key，服务可以识别重复请求，并保证同一事实只落一次流。

circuit breaker（熔断器）是发现事件速率异常或循环时暂停流的保护机制。视频里的阈值直觉是每秒超过约 100 个事件就触发暂停。它要防的场景很朴素：processor A 看到事件后 append 给 B，processor B 又 append 给 A，两个插件互相激发，stream 在几秒内膨胀到几千条事件。熔断器保留历史，同时追加或设置 paused 状态，让人或管理程序显式 resume。

pause/resume 也是协议的一部分 暂停 stream 采用公开协议表达，而非数据库管理员在后台改一行隐藏状态。视频里的接口是 append 一个表示 paused 的事件，通常还要给出 reason，例如 demo。resume 也通过事件表达。这样做的好处是审计清楚：谁在什么时候因为什么暂停了流。风险也直接：如果权限设计粗糙，攻击者可以 append pause 事件让 agent 停摆，所以生产系统必须把 pause/resume 放进认证、授权和审计边界。

2.5 schedule、heartbeat 与 push/pull subscription

事件流还可以承载时间。Jonas 展示了 heartbeat：通过 append 一个 schedule 相关事件，让系统每 5 秒生成一次 heartbeat event。也可以安排 10 分钟后的事件、某个具体时间点的事件，或者取消 schedule。这里的机制很值得注意：调度规则本身也作为事件进入 stream，未来触发的 heartbeat 同样作为事件出现。

这让时间从“外部 cron 黑盒”变成可审计事实。比如一个 agent 每 5 秒检查一次长任务状态，heartbeat event 可以驱动 processor 做检查；如果用户后来取消 schedule，取消动作也留在流里。边界是精度和重复投递：真实调度系统可能受到延迟、重启、网络抖动影响，因此 heartbeat 消费方仍然要用 offset、created_at 和 idempotency key 处理重复或迟到事件。

push subscription（推送订阅）与 **pull subscription**（拉取订阅）是两种消费事件的方向。pull subscription 是客户端主动连接 HTTP 服务端，拿到 SSE response stream；视频中的终端 curl 就是 pull，客户端负责维持连接、断线重连、记录 offset。push subscription 是反过来：stream 服务在事件追加后，把事件或过滤后的事件推到指定 endpoint，例如某个 server、第三方服务、Slack API endpoint。

push 与 pull 的选择 pull 更适合本地开发、调试和长时间运行的 processor，因为客户端掌握重连节奏，也容易从指定 offset 恢复。push 更适合外部 webhook 风格集成：你的服务暴露 endpoint，事件流服务主动通知。push 的风险面更大，必须处理 endpoint 不可达、重试风暴、签名校验、幂等去重；pull 的主要成本是保持连接和管理消费进度。

到这里，Section 2 的最小原语已经闭合：path 决定流，event envelope 统一事件形状，offset 给出顺序坐标，SSE 和 subscription 让处理器观察变化，idempotency key、pause/resume、circuit breaker、schedule/heartbeat 则把可靠性和时间控制继续事件化。

2.6 本章小结

本章建立了 events.iterate.com 的基础操作模型。event stream 是同一路径下的有序事实序列；event envelope 至少包含 type，通常带 payload，并由服务端补齐 path、offset、created_at；offset 从 1 开始增长，是重放、订阅和调试的坐标。?live=true 通过 SSE 让连接保持打开，新的 append 会持续被观察者收到。

更重要的是，错误、幂等、暂停、熔断和调度都被放回同一种事件模型里。无效 payload 产生 invalid event appended，webhook 重试靠 idempotency key 去重，pause/resume 用事件管理流状态，超过约 100 events/second 的循环风险由 circuit breaker 截断，heartbeat 与未来任务通过 schedule 事件表达。下一章会把这个“所有东西都是事件”的接口面继续收缩，说明为什么 token chunk、工具调用、错误和用户输入也可以进入同一条流。

3 一种 API 视角：所有 Agent 行为都落成事件

3.1 问题：插件 API 为什么会越长越难用

在 00:16:39–00:20:20 这一段，提问者问的是一个很直接的问题：这个系统的架构到底是什么？Jonas 的回答很短：核心几乎就是 API。他把现有 agent 插件体系拿来对照，例如 OpenCode extension、Pi extension、Claude extension。这些体系通常会暴露一组专门接口：流式输出有一个接口，工具调用有一个接口，错误处理有一个接口，用户输入又有一个接口。接口越多，插件作者越需要学习每个入口的时序、权限、重试和失败语义。

这一章的架构命题是：把 API surface 收缩到 **event**。API surface 指系统暴露给外部开发者的可调用边界，就像一台机器露在外面的按钮、旋钮和插孔。按钮越多，能做的事可能越多，误触和兼容成本也越高。这里的收缩策略很朴素：外部世界只追加事件、订阅事件、解释事件。Agent harness 的每个动作都先落成事件，再由后续 processor、UI、审计器或外部服务消费。

本章核心判断 本节的关键超出“事件日志可以记录 Agent 行为”这个常识，落在“Agent 行为的外部接口可以统一压缩成事件”这一层。Streaming chunks、tool call、error、user input 都使用同一种 envelope 进入 event stream 后，扩展点就从一组框架私有 hook 变成一条可订阅、可重放、可组合的事实流。

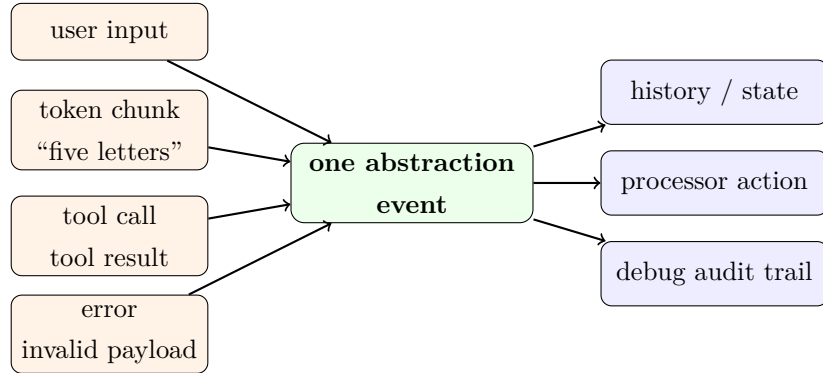
用形式化语言写，若某个 Agent 运行过程中的可观察行为集合为

$$B = \{b_{\text{input}}, b_{\text{chunk}}, b_{\text{tool}}, b_{\text{error}}, b_{\text{state}}\},$$

那么本章的抽象函数是

$$\phi: B \rightarrow E, \quad \phi(b_i) = e_i = (\tau_i, p_i, o_i, c_i).$$

其中 τ_i 是事件类型， p_i 是 payload， o_i 是流内 offset， c_i 是创建时间。上一章已经解释 event envelope 和 offset，本章只使用这个模型来说明 API 归一化：系统把不同种类的行为 b_i 映射到同一种事件对象 e_i 。

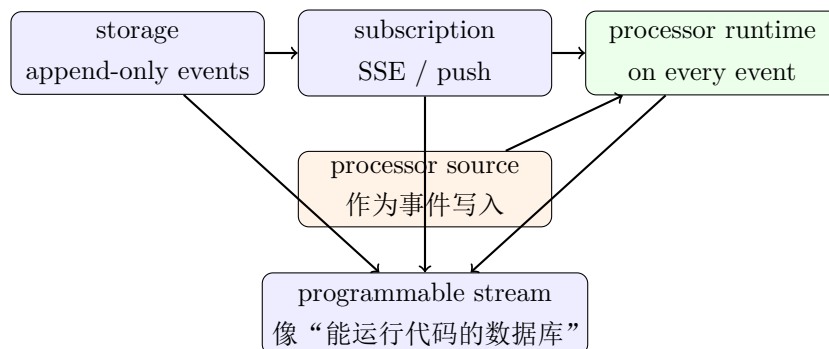


API surface 收缩到一个核心对象后，插件只需要声明自己理解哪些 event type。

图 6: 把 chunks、tool、error、input 统一落成 event

6

⁶视频画面时间区间：00:17:11–00:18:45。



stream 服务同时承担事实存储、事件分发、处理器唤醒，适合快速实验 agent harness。

图 7: programmable stream: 存储、订阅、处理器三合一

7

3.2 机制: streaming chunk、tool call、error、user input 都可事件化

Jonas 举的第一个例子是 partial completion 的 streaming chunks。许多 Agent SDK 会把“模型正在吐出一小段 token”设计成特殊流接口；这里可以把它表达成普通事件，例如“收到 5 个字符的 streaming response”。这件事听起来小，工程后果很大：UI 可以订阅这些 chunk 事件来逐字渲染，审计系统可以保留完整生成轨迹，测试系统可以在某个 offset 截断并复现场。

同样，tool call 也可以事件化。一次工具调用可以拆成几类事实：模型请求调用某个工具、参数已解析、外部 API 已发起、工具返回成功、工具返回错误。这里无须提前引入具体 reducer 代码；只需要看到接口形态：工具调用并非框架内部的黑箱回调，它可以是一串可观察事件。对 Agent 来说，工具调用像一张物流单：下单、出库、运输、签收、异常，每一步都能盖章入账。

错误也应进入同一个模型。第二章已经出现过无效 payload 触发错误事件的例子；本章把这个原则扩展到 Agent 行为层面。LLM provider 报错、工具超时、payload schema 不匹配、processor 执行失败，都可以形成 error event。错误事件的价值在于它能被后续 processor 看见：告警 processor 可以通知人，修复 processor 可以尝试生成补丁，UI processor 可以把错误投影成 feed item。

用户输入同样适合事件化。一个 Web form 提交、一条 Slack webhook、一段命令行输入，在事件流里都可以成为 agent input added 一类事件。输入事件的 payload 可以非常小，甚至只是一段字符串；复杂系统再逐步给它附加来源、权限、语言、上下文窗口位置等元数据。这里的思路像会计里的“凭证”：无论现金、转账、退款还是折扣，先变成账本可处理的凭证，再进入统一流程。

事件化与普通日志的差别 普通日志常常是旁路记录：程序先做事，随后写一行给人看。事件化要求系统把“发生了什么”作为主数据路径：先把事实追加到 event stream，状态、UI、外部副作用再围绕事件推进。直观类比是“监控录像”和“银行流水”的差别：录像能回看，流水能结算、对账、追责和重建余额。

⁷视频画面时间区间：00:18:16–00:19:18。

3.3 定义：programmable stream 是什么

Jonas 在回答中给出一个很重要的比喻：events iterator app 可以看成存储层，几乎像数据库；再进一步，它给人一种“在数据库里运行 TypeScript”的错觉。他提到 Convex，是因为 Convex 让开发者感觉业务函数贴近数据运行。这个视频里的更准确说法是 **programmable stream**。

programmable stream 的正式定义 **Programmable stream (可编程流)** 是一种把 append-only event log、订阅分发机制和 processor 执行能力组合在同一个基础设施边界内的系统形态。它至少包含三件事：第一，按 path 和 offset 持久化事件；第二，让本地程序、Web service 或平台 worker 订阅新事件；第三，允许 processor 在事件到达后读取状态、执行受控副作用并追加新事件。用符号表示，若事件序列为 $E = \langle e_1, \dots, e_n \rangle$ ，processor 集合为 $\mathcal{P} = \{P_1, \dots, P_m\}$ ，则 programmable stream 提供的运行语义近似为

$$P_j(E_{1:n}) \rightarrow \Delta E_j,$$

其中 ΔE_j 是该 processor 观察既有事件后追加的新事件集合。

这个定义里，“stream”保留了流的时间性：事件按 offset 单调推进，新的事实不断追加；“programmable”说明流服务承担了一部分执行编排：它能唤醒外部 API、连接程序、触发 processor，后续章节还会讲到把 processor 源码作为事件写入流后自动运行的形态。本章只给出概念边界，配置事件的 payload 结构留到第七章。

这也解释了 events iterator app 为什么同时像存储层和 programmable stream。它像存储层，是因为它持久保存 raw events，提供路径、offset、created_at 这些数据库式坐标；它像 programmable stream，是因为它可以把每个新事件交给 processor，让 processor 在真实互联网事件上做实验：响应 Webhook、调用外部 API、追加结果事件、被 UI 投影出来。

3.4 边界：TypeScript 是 workshop 选择，目标是 polyglot

00:19:18 以后，Jonas 转向 TypeScript 示例。他先问现场有没有人不会 TypeScript，然后立刻补了一句：这个架构原则上是 polyglot。原因很简单：核心 API 很小，可以通过 OpenAPI spec 生成不同语言的 client。TypeScript 和 npm package 只是 workshop 的教学路径，方便现场快速演示 SDK；架构目标并不依赖 TypeScript 运行时。

Polyglot 的意义在分布式 Agent harness 里尤其具体。一个安全检查 processor 可以用 Rust 写，一个 RAG processor 可以用 Python 写，一个 UI projection 可以用 TypeScript 写，一个内部审计服务可以用 Go 写。只要它们都能追加事件、订阅事件、理解事件类型，它们就能在同一条 event stream 上协作。这里的语言关系像不同国家的人使用同一套报关单：各自国内流程不同，关口识别的字段一致即可。

接口变小并不自动带来安全 把 API surface 收缩到 event 会降低集成复杂度，也会把风险集中到事件入口。公开 stream、外部 webhook、错误事件、工具调用结果、用户输入都进入同一条流，意味着认证、授权、schema 校验、速率限制和来源证明（client provenance, Section 8 详解）必须跟上。否则，攻击者只要能写入事件，就可能驱动整个 Agent harness 的后续行为。

这一章还需要守住两个边界。第一，本章不写具体 reducer 代码；stream processor 如何把事件归约成状态、如何把副作用放到 after-append 阶段，是第四章的职责。第二，本章不展开动态 worker 的配置细节；“源码可以放入事件并在每个事件上运行”这里只作为 programmable stream 的能力预告，具体 payload、secret 注入和 bundle 限制留到第七章。

3.5 本章小结

本章把问答段重组为一个 API 设计命题：Agent harness 的外部接口可以尽量收缩到 event。Streaming chunks 可以是“收到若干字符”的事件，tool call 可以是一串请求、执行、返回、失败事件，error 可以进入同一条可审计流，user input 也可以作为最小输入事实追加。这样一来，events iterator app 既承担存储层职责，又呈现 programmable stream 的形态：保存事件、分发事件、唤醒处理器、让处理器继续追加事件。TypeScript 只是本次 workshop 的实现选择；真正的目标是让不同语言写成的 processor 通过同一条事件流协作。

4 从 curl 到 TypeScript: stream processor 的 reducer/after-append 分工

前面几章已经把事件流的外壳讲清楚了：一条路径对应一条 stream，所有事实按 offset 追加进去，`?live=true` 可以把后续事件以 SSE 的形式持续推给客户端。本节把视角从 curl 推进到 TypeScript SDK。讲者在 00:20:19 之后试图让现场观众拉取 workshop 仓库，运行一个最小例子，再从这个例子过渡到真正的 stream processor。现场环境并不稳定，编辑器、类型服务和网络都有卡顿，因此本节只把可验证的机制抽出来讲清楚，不把这段演示写成完整跑通的产品教程。

这段失败仍有一个有效信号：它并没有证明事件流模型失效。字幕里同时出现了几类现场故障：AI Engineer Workshop 仓库里刚修过 client-side type error，讲者的 Cursor/TypeScript language server 没有识别类型，运行时还报过 `invalid TypeScript syntax`；但现场有参与者能够跑起 example TS，并能发送 ping 得到 pong。因此本节后面的判断应落在“最小 ping-pong 闭环可验证，OpenAI agent 现场组装不稳定”，而不应把失败归因到 stream processor 抽象本身。

4.1 从 SDK 调用看见同一条事件流

TypeScript SDK 做的第一件事，是把前面手写的 HTTP 请求包成三个动作：

- `createEventsClient`：创建访问事件服务的客户端，配置 base URL、path prefix、项目命名空间等参数。

- **stream**: 连接某条 stream, 通常带上 `live: true`, 效果等价于前面用 `curl -N` 打开的 SSE 长连接。
- **append**: 向某条 stream 追加事件, 例如把一个 `ping` 输入变成后续的 `pong` 输出。

这一层没有引入新的数据库概念。SDK 只是把“路径、事件、追加、订阅”换成了更适合程序组合的函数接口。直观类比是: 前面的 `curl` 像手动往账本上写一笔流水, SDK 像给账本装上一个小型柜台程序; 柜台程序依然只能追加事实, 不能悄悄改写历史。

```
1  const client = createEventsClient({
2    baseUrl: "https://events.iterate.com",
3    pathPrefix: "/workshop/alice",
4  });
5
6  for await (const event of client.stream({
7    path: "/demo",
8    live: true,
9  })) {
10    if (event.type !== "ping") continue;
11
12    await client.append("/demo", {
13      type: "pong",
14      payload: { inReplyTo: event.offset },
15    });
16  }
```

Listing 1: SDK 层面的 ping-pong 轮廓, 省略认证与错误处理

这个例子表达的是最小反应式行为: 监听 `ping`, 追加 `pong`。它已经是 agent harness 的雏形, 因为系统的输入、判断和输出都留在事件流里。后面复杂的 LLM 请求、工具调用、子 agent 通信, 只是在这条链路上增加更多事件类型和状态判断。

本节的关键分工是: **stream** 负责把新事件送到本地程序, **append** 负责把程序产生的新事实写回流里; stream processor 则规定“状态计算”和“外部副作用”各自放在哪里。没有这个分工, 重启、补进度和重复请求会混在一起, 最终很难判断一次 LLM 调用到底由哪个事件触发。

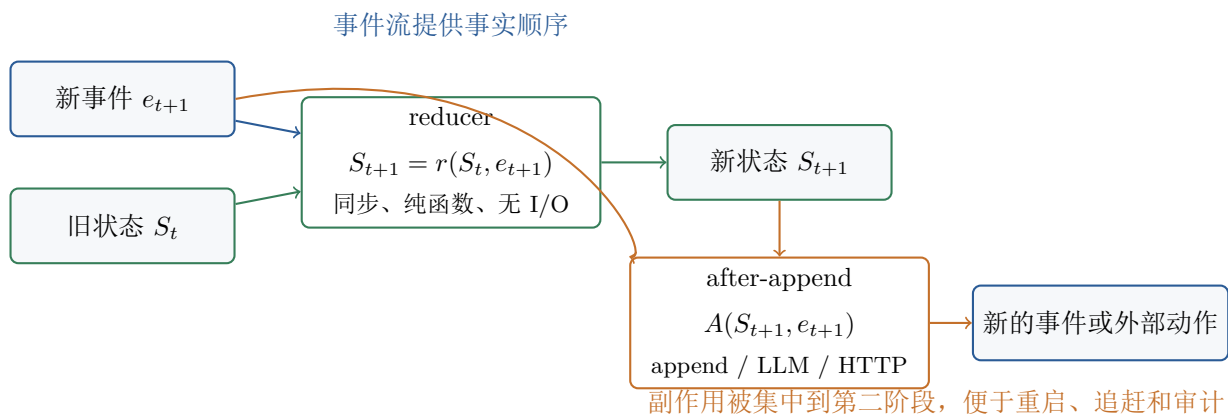
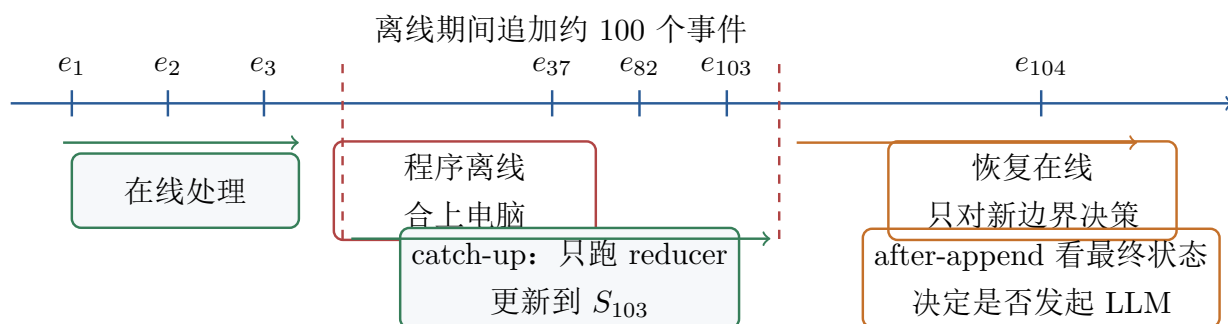


图 8: stream processor 把状态归约和副作用拆成两个阶段: reducer 只把事件折叠成状态, after-append 再根据最终状态触发 append、LLM 请求或外部 I/O。⁸



关键边界: 历史事件用于补状态, 外部副作用只在恢复后的决策点发生。

图 9: 重启 catch-up 时间线: 程序离线期间进入流的旧事件先被归约进状态, 随后才判断是否需要新的副作用, 避免把历史事件逐个重放成昂贵的 LLM 请求。⁹

4.2 正式定义: stream processor、reducer 与 after-append

流处理器 (stream processor) 是订阅一条或多条 event stream、按事件顺序维护派生状态、并在必要时触发副作用的程序单元。它像一个一直盯着账本的会计: 每来一笔流水, 先更新余额、分类和统计; 如果余额触线, 再发通知或追加一条新流水。

归约器 (reducer) 是 stream processor 内部的同步状态函数。它接收当前状态和一个新事件, 返回新状态。形式化写作:

$$S_{t+1} = r(S_t, e_{t+1})$$

其中:

- S_t : 处理到 offset t 之后的派生状态, 例如 “已经看见多少个 hello world 事件” “最近一次 ping 的 offset” “当前是否有待处理的 LLM 请求”。
- e_{t+1} : 下一条事件, 包含 type、payload、stream path、offset、created_at 等信封字段。

- r : reducer 函数, 读取 S_t 和 e_{t+1} , 同步计算 S_{t+1} 。
- S_{t+1} : 应用该事件后的新状态; 如果事件类型无关, S_{t+1} 可以等于 S_t 。

数学上, reducer 把事件序列 $E = \langle e_1, e_2, \dots, e_n \rangle$ 折叠成状态:

$$S_n = r(\dots r(r(S_0, e_1), e_2) \dots, e_n)$$

这就是事件溯源的可重建性。只要初始状态 S_0 和事件序列 E 相同, reducer 得到的 S_n 就应相同。为保证这一点, reducer 必须像纯计算一样运行: 不发 HTTP 请求, 不调用 LLM, 不写数据库, 不追加事件, 不依赖当前墙钟时间来改变结果。

“归约”这个词来自函数式编程中的 reduce/fold: 把一串值压成一个汇总值。数组求和是最小例子: $sum_{t+1} = sum_t + x_{t+1}$ 。stream processor 的 reducer 也是同一件事, 只是 x 变成了事件 e , 汇总值变成了业务状态 S 。

追加后钩子 (after-append hook) 是在事件已经写入、reducer 已经更新状态之后运行的副作用阶段。它的形式化写法可以记为:

$$A(S, e) \rightarrow \{\alpha_1, \dots, \alpha_k\}$$

其中:

- A : after-append hook, 读取最新状态 S 和刚处理的事件 e 。
- α_i : 一个副作用动作, 例如 **append** 新事件、调用 LLM、**fetch** 外部 API、发送通知。
- k : 这次事件触发的动作数量; $k = 0$ 表示无需副作用。

如果 reducer 是“更新账本余额”, after-append 就是“余额触线后给银行发短信”。短信可能失败、延迟、重试, 也可能产生新的事件; 这些不确定性都应留在 after-append, 不应污染 reducer 的可重放性。

4.3 ping-pong processor: 最小可工作的分工

讲者在现场用 ping-pong processor 说明这套分工: 当 stream 中出现 **ping**, processor 追加 **pong**。更完整的写法可以把“记录状态”和“产生副作用”拆开:

```
1 type State = {
2   lastPingOffset: number | null;
3   shouldReply: boolean;
4 };
5
6 const initialState: State = {
```

```

7   lastPingOffset: null,
8   shouldReply: false,
9 };
10
11 function reducer(state: State, event: Event): State {
12   if (event.type !== "ping") {
13     return { ...state, shouldReply: false };
14   }
15
16   return {
17     lastPingOffset: event.offset,
18     shouldReply: true,
19   };
20 }
21
22 async function afterAppend(ctx: Context, state: State, event: Event) {
23   if (!state.shouldReply) return;
24
25   await ctx.client.append(ctx.streamPath, {
26     type: "pong",
27     payload: { inReplyTo: state.lastPingOffset },
28   });
29 }

```

Listing 2: TypeScript-like processor: reducer 只算状态，afterAppend 才追加事件

这段伪代码故意短小，因为它要表达的是边界。reducer 面对 ping 只把状态改成“需要回复”；afterAppend 看到这个状态后，才调用 append。如果未来把 append 换成 LLM 请求、Slack API、Webhook 或数据库写入，边界仍然相同：同步归约阶段负责判断事实，副作用阶段负责同外部世界交互。

不要在 reducer 里调用 LLM 或外部 API。reducer 会在重放历史、catch-up、测试、UI 投影和其他 processor 复用中反复运行。把网络 I/O 塞进去，会把“重新计算状态”变成“重新花钱、重新发消息、重新改外部系统”。

4.4 processor runtime: 把事件流接到处理器

processor runtime 是连接层。它负责订阅 stream、读取历史或新事件、按 offset 顺序调用 reducer，并在合适的时机调用 after-append。可以把它理解成一个轻量调度器：

1. 启动时读取 stream，从已知 offset 或初始 offset 开始。
2. 对每个事件调用 $S_{t+1} = r(S_t, e_{t+1})$ ，把状态推进到最新。
3. 当事件属于“当前在线处理窗口”时，调用 $A(S, e)$ 产生副作用。

4. 副作用如果追加了新事件，新事件再次进入同一条事件流，形成可审计的闭环。

这里的“runtime”不等于某个庞大的服务器框架。演讲中的部署方式有三种雏形：本地电脑用 SSE 订阅运行、普通 Web service 接收推送订阅运行、平台上的 dynamic worker 自动运行。本节只关注本地 runtime 的抽象：它把原始 event stream 接入 processor，让开发者写 reducer 和 after-append，少写订阅、取消、AbortSignal、错误恢复这些样板代码。

4.5 重启 catch-up: 旧事件可以重算，旧副作用不能乱放

讲者在 00:31:46 给出了这个设计最现实的理由：程序会离线。今天的 workshop 例子跑在笔记本上，用户合上电脑，期间 stream 里可能新增 100 个事件；明天重新打开程序时，processor 必须追上这些事件，否则状态会落后。

catch-up 的正确动作是：对离线期间的旧事件运行 reducer，把 S_t 推进到最新 S_n 。这一步像把账本从第 t 行补算到第 n 行，目标是获得当前余额。after-append 的动作要更谨慎：如果这些旧事件中有 20 条“需要调用 LLM”的输入，重启后直接补发 20 次 LLM 请求通常就是事故，轻则浪费 token，重则给用户发送过期回复，甚至触发工具调用的连锁副作用。

因此 runtime 需要区分两类工作：

- **状态追赶**：历史事件全部进入 reducer，保证当前状态完整。
- **副作用触发**：after-append 只在满足运行时策略时执行，例如只处理最新事件、只处理未完成任务、或者用状态判断某个请求仍然有意义。

这个边界也解释了为什么 LLM agent 不能只写成“看到用户输入就立刻请求模型”的循环。事件溯源系统里的输入会被重放，重放用于恢复状态；模型请求属于外部副作用，必须带上幂等、offset、水位线或任务状态判断。形式化地说， r 应该总是可重复执行； A 需要受 runtime 策略约束，尤其要知道当前处理的是历史 catch-up 事件还是 live 事件。

4.6 本章小结

本节把 curl 层面的事件追加推进到了 TypeScript 层面的 stream processor。createEventsClient 创建客户端，stream 订阅事件，append 写回新事实；ping-pong processor 用最小例子证明“监听事件、更新状态、追加响应事件”这条链路足够表达行为。

更重要的是三件事的分工已经正式确定：stream processor 是订阅事件流并维护派生状态的程序单元；reducer 是同步纯函数，用 $S_{t+1} = r(S_t, e_{t+1})$ 把事件折叠成状态；after-append hook 用 $A(S, e)$ 产生 append、LLM、API I/O 等副作用。这个设计的硬边界是重启 catch-up: 旧事件可以重算状态，旧副作用必须受控触发。后面构造 LLM agent 时，历史、系统提示、模型响应和工具调用都会落回这条分工线上。

5 把 LLM Agent 状态化：输入事件、历史、系统提示与响应流

上一章已经把 stream processor 的基本骨架拆开：reducer 负责把事件归约成状态，after-append 负责调用外部系统并追加新事件。本章进入 00:32:46–00:44:17 这段最关键的 LLM agent 建模：如果一条事件流要变成一个会响应用户的 agent，它需要哪些状态，哪些输入事件会触发 LLM 请求，OpenAI Responses API 的消息结构又怎样从本系统的事件模型翻译出来。

讲者现场试图构造一个 OpenAI agent processor。代码没有稳定跑通，仓库示例也在现场被重写过，部分片段明显处于半成品状态。这里要抓住能成立的概念：事件模型、路径层级、ping-pong 式 processor 抽象、reducer/after-append 分工。这些部分已经由前面的简单 processor 证明；OpenAI agent processor 的现场实现只能当作架构草图，不能当作可复制的生产代码。

5.1 子路径与 sub-agent：把路径当作协作空间

讲者从一个容易被忽略的能力开始：processor 不只能向当前路径追加事件，也可以向子路径、父路径或其他路径追加事件。事件服务里的 path 像文件系统路径，/alice/main 可以有 /alice/main/boris 这样的子路径。这个层级结构让 sub-agent 变得很自然：父 agent 不需要新建一套特殊 RPC 协议，只要向子路径追加输入事件，再订阅子路径上某些结果事件。

例如父 agent 想将一个任务委派给名为 Boris 的子 agent，可以向 ./boris 追加一条 agent input added：

```
1 await client.append("./boris", {
2   type: "agent_input_added",
3   payload: "Summarize the latest user request and return an action plan."
4 });
```

Listing 3: 用子路径表达 sub-agent 委派，省略认证和错误处理

Boris 的 processor 订阅 ./boris，处理完后追加一条结果事件，例如 agent result added。父 agent 订阅这个结果事件，就能继续自己的状态归约。直观类比是办公室里的文件夹：主任务文件夹下面放一个 Boris 子文件夹，父任务把便签放进去，Boris 把处理结果放回文件夹；所有便签都按时间顺序留档。

sub-agent 的核心接口仍然是事件。父 agent 发起委派，表现为对子路径 append 输入事件；子 agent 完成工作，表现为对子路径 append 输出事件；父 agent 被唤醒，表现为订阅到输出事件后继续 reducer/after-append 流程。这里没有隐藏的函数调用栈，也没有不可审计的后台消息。

这个做法的代价也很直接：路径越多，processor 越多，循环和竞态越容易出现。父 agent 等 Boris，Boris 又把任务交回父 agent，就可能产生事件风暴。前面章节提到的 pause、resume、circuit breaker，在这种多 agent 路径树里会变成硬需求。

5.2 Agent state: model、system prompt、history 与运行 flag

进入 LLM agent 之后，问题变成一句朴素的工程话：发起一次 LLM 请求之前，processor 到底需要知道什么？讲者在 00:38:25 后把答案缩到三个基础字段：

- **model**：要调用的模型名称，例如现场口头举例里的 GPT 系列模型。它是状态字段，因为模型可以由事件改变。
- **systemPrompt**：系统提示，也就是 Responses API 里常见的 instructions。讲者认为 instructions 这个名字容易误导，所以在 agent state 里更愿意称它为 system prompt。
- **history**：历史消息序列，用来告诉模型此前发生了什么。它属于从事件日志派生出来的 LLM 输入视图，粒度小于完整日志。

形式化地写，一个极简状态可以表示为：

$$S = \{\text{model} = m, \text{systemPrompt} = p, \text{history} = h\}$$

其中 m 是模型名， p 是系统提示， h 是历史消息序列。更具体地，历史写作：

$$h = [m_1, \dots, m_k]$$

这里的 m_i 指第 i 条 message，含义不同于模型字段 model。每条 message 通常包含角色、内容和可能的元数据。这个符号强调一件事：LLM 看到的“对话历史”是一个有序数组，顺序不能随便改。就像病例记录或法庭证词，前后顺序会改变解释空间；同一句话出现在用户提问前和模型回答后，含义完全不同。

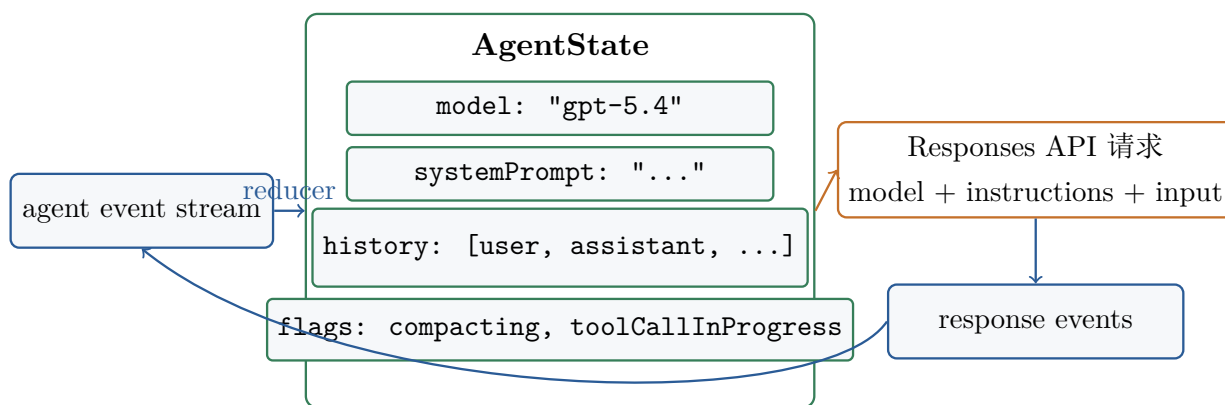
初始状态可以很小：

```
1 type AgentState = {
2   model: string;
3   systemPrompt: string;
4   history: Array<{ role: string; content: string }>;
5   compacting: boolean;
6   toolCallInProgress: boolean;
7 };
8
9 const initialState: AgentState = {
10   model: "gpt-5.4",
11   systemPrompt: "You are a helpful event-sourced agent.",
12   history: [],
13   compacting: false,
14   toolCallInProgress: false,
15 };
```

Listing 4: 极简 LLM agent state

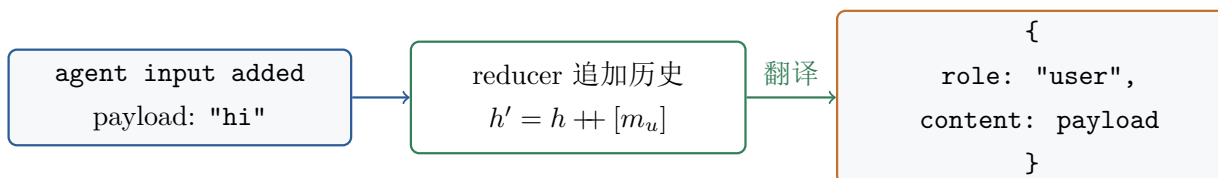
`compacting` 和 `toolCallInProgress` 是讲者点名的可扩展 flag。它们看起来像小字段，实际代表两类重要状态：上下文压缩与工具调用并发。上下文压缩用于控制历史过长的问题；工具调用 flag 用于避免同一时间启动多个互相踩踏的工具请求。事件溯源的优势在这里很明显：新增一个能力通常对应新增事件类型和几行 reducer，避免把一团隐式变量塞进运行时内存。

history 与 event log 是两层东西。event log 是完整事实流水，包含用户输入、模型响应片段、错误、工具调用、配置变更；history 是喂给 LLM 的消息视图，通常只挑选一部分事件并翻译成模型 API 期望的结构。用数据库类比，event log 像原始交易表，history 像一张专门给模型查询的投影表。



状态像账本余额：它不存“怎么来的感觉”，只存事件折叠后的可用事实。

图 10: LLM agent state 的最小骨架：模型名、系统提示、历史消息和运行 flag 都由事件流归约得到，after-append 再把这些字段组装成一次 LLM 调用。¹⁰



本系统事件	状态历史	Responses API 消息
type payload	history.push 用户输入成为一条消息	role/content 符合 OpenAI 输入结构

图 11: agent input added 到 Responses API 的映射：自家事件可以只保存字符串，但进入 OpenAI 调用前必须变成带 role 和 content 的消息结构。¹¹

5.3 输入事件：agent input added 极简到只有字符串

现场 UI 里有一个“cheat code”，可以快速创建 agent input added 事件。讲者强调它故意设计得非常简单：payload 可以只是一个字符串。用户说“hi”，事件就可以长这样：

```

1 {
2   type: "agent_input_added",
3   payload: "hi"
4 }

```

Listing 5: 最小用户输入事件

这个极简输入有两个好处。第一，外部系统接入容易，Web form、Slack webhook、命令行脚本都能把文本追加进事件流。第二，reducer 的职责清楚：看到这类事件，就把它变成 history 里的一条用户消息。输入事件本身不需要提前知道 OpenAI API 的全部字段，也不需要携带模型供应商专有结构。

用数学记号表达，设用户输入文本为 x_u ，本系统的输入事件可以抽象成：

$$e_u = (\tau = \text{agent input added}, \text{payload} = x_u)$$

reducer 处理它时，并不会调用模型。它只更新状态中的历史：

$$h' = h \mathrel{++} [\text{toUserMessage}(x_u)]$$

这里的 $++$ 表示数组拼接，toUserMessage 是把本系统输入 payload 翻译成模型消息对象的函数。这个动作必须放在 reducer 里，因为“用户输入已经加入历史”是可重放、可验证的状态变化。

5.4 Responses API role/content 映射：从自家事件到模型消息

OpenAI Responses API 期望输入消息带有 role 和 content。事件流里的用户输入只有一个字符串，模型 API 需要一个结构化对象。因此现场代码的关键映射可以写成：

$$x_u \mapsto \{\text{role} : \text{user}, \text{content} : x_u\}$$

这个公式的含义很具体：

- x_u ：用户输入事件的 payload，例如字符串 "hi"。
- $\text{role} : \text{user}$ ：告诉模型这条消息来自用户，而非系统、助手或工具。
- $\text{content} : x_u$ ：把原始文本放到 content 字段中，成为 Responses API 可以消费的消息内容。

对应的 reducer 轮廓如下：

```

1 function reducer(state: AgentState, event: Event): AgentState {
2   if (event.type !== "agent_input_added") return state;
3 }

```

```

4   const userText = event.payload;
5
6   return {
7     ...state,
8     history: [
9       ...state.history,
10      { role: "user", content: userText },
11    ],
12  };
13 }

```

Listing 6: 把 agent input added 归约成 OpenAI 风格 history

这个翻译层很关键。事件模型是系统内部的事实语言，Responses API 是模型供应商的请求语言。二者需要明确边界。今天可以把 agent input added 翻译成 OpenAI 的 {role, content}; 明天换成别的模型供应商，可以保留事件类型和历史归约逻辑，只替换 after-append 中的 API 适配层。

不要把供应商 API 的完整请求体当作内部事件格式。那会让事件日志被某个 SDK 版本、某个模型厂商、某个字段命名长期绑定。更稳的做法是让事件表达业务事实，例如“用户输入已加入”“模型输出片段已产生”，再在边界处翻译成 Responses API 所需结构。

5.5 after-append: 用状态发起 LLM 请求

当 reducer 把用户输入追加到 h 后，after-append 才有资格考虑调用 LLM。一次 Responses API 请求至少要使用三个状态字段：

```

1  async function afterAppend(ctx: Context, state: AgentState, event: Event) {
2    if (event.type !== "agent_input_added") return;
3    if (state.compacting || state.toolCallInProgress) return;
4
5    const stream = await openai.responses.create({
6      model: state.model,
7      instructions: state.systemPrompt,
8      input: state.history,
9      stream: true,
10   });
11
12   for await (const responseEvent of stream) {
13     await ctx.client.append(ctx.streamPath, {
14       type: "openai_response_event_added",
15       payload: responseEvent,
16     });
17   }
18 }

```

Listing 7: after-append 使用 state 构造 Responses API 请求

这段代码展示的是责任分配，细节不能照抄成生产实现。真实系统还要处理 API key 注入、幂等键、重启后是否重放请求、stream 中断、最终 response 聚合、工具调用、错误事件和速率限制。现场 OpenAI agent processor 的代码正是在这些细节上不稳定，所以讲者后来承认这部分演示失败。

概念上，after-append 的输入是 S 和刚追加的事件 e ，输出是一组副作用动作：

$$A(S, e_u) \rightarrow \{\text{LLM}(\text{model} = m, \text{instructions} = p, \text{input} = h)\}$$

如果 e_u 是用户输入事件，且当前没有 compacting、tool call in progress 等阻塞 flag，processor 才发起模型请求。模型请求产生的流式响应随后被追加回同一条事件流，形成可审计闭环。

5.6 响应事件与 history 归约：流式输出也要落账

Responses API 的 streaming 模式会吐出很多 response event。有些是输出文本片段，有些是状态变更，有些可能表示完成、错误或工具调用。讲者在 00:43:47 附近开始解释这一点，但现场代码很乱，他自己也直接评价“这有点胡扯”。这里需要把概念补齐：只要模型产生了可观察事实，就应该追加事件。

一种合理的事件拆分是：

- **response output text delta added**: 模型吐出一小段文本。
- **response completed**: 一次模型响应结束，带最终 response id、usage 或 finish reason。
- **response failed**: 模型调用失败，带错误类型和可展示消息。
- **tool call requested**: 模型要求调用工具，带工具名和参数。

reducer 可以用这些响应事件把 assistant 消息归约进 history。若模型分三段吐出 $\delta_1, \delta_2, \delta_3$ ，可以先把片段暂存在状态里：

$$y = \delta_1 \parallel \delta_2 \parallel \delta_3$$

其中 $\parallel$ 表示字符串拼接。当 **response completed** 到来时，reducer 再把完整 assistant 消息追加到历史：

$$h' = h \mathrel{++} [\{role : assistant, content : y\}]$$

这个归约策略有一个实际好处：UI 可以实时显示 delta 事件，history 可以只保存完成后的 assistant message。两者都来自同一条事件流，却服务不同读模型。UI 关心“现在屏幕上逐字出现什么”，LLM 下一轮请求关心“上一轮完整回答是什么”。

还要注意失败和取消。模型请求中断时，事件流里应该有 `response failed` 或 `response cancelled`，reducer 据此清理 `toolCallInProgress`、`compacting` 或临时 buffer。否则下一次输入可能被旧 flag 卡住，agent 表面上像“没反应”，实际是状态机停在半路。

5.7 现场边界：思想成立，代码不稳定

这段演示最容易被误读。讲者确实想现场搭出 OpenAI agent processor：定义 state，定义 agent input added schema，用 reducer 更新 history，再用 Responses API streaming 追加响应事件。但从 00:37:55 开始，他已经承认示例没跑起来；到 00:43:47，响应事件处理片段也明显没有整理完成。

所以本章能确认的内容并非“这个 OpenAI processor 已经成熟”。可确认的是以下几条架构命题：

- ping-pong processor 证明了事件进入、状态更新、追加输出的最小闭环。
- path 层级证明了 sub-agent 可以被建模为子路径上的 processor。
- LLM 请求所需的 model、system prompt、history 可以从事件流归约成 state。
- 用户输入事件可以保持业务语义，再翻译成 Responses API 的 role/content 消息。
- 流式响应可以作为 response event 回写，随后被 reducer 归约成 assistant history 或 UI 投影。

这也是事件溯源 agent harness 的严酷现实：抽象很干净，工程实现会马上遇到 SDK 版本、schema、streaming 事件类型、错误恢复和重启幂等问题。现场失败属于架构边界提示，它提醒读者不要把 demo 代码当作系统可靠性的证据。可靠性来自清楚的事件边界、可重放 reducer、受控 after-append 和完整错误事件。

5.8 本章小结

本章把 LLM agent 拆成了可归约状态和可追加事件。子路径让 sub-agent 像文件夹一样自然派生；agent state 至少包含 model、systemPrompt 和 $h = [m_1, \dots, m_k]$ 这样的历史消息序列，并可继续扩展 compacting、toolCallInProgress 等运行 flag。

agent input added 可以极简到只含字符串 x_u 。reducer 把它翻译成 $x_u \mapsto \{role : user, content : x_u\}$ ，追加进 history；after-append 再用 state 构造 Responses API 请求。模型 streaming 输出继续以响应事件写回事件流，reducer 再把片段归约为 assistant history。现场 OpenAI agent processor 代码不稳定，但事件模型与 ping-pong/processor 抽象已经足够支撑这套设计。

6 失败的现场 demo 反而暴露了真实架构：内置 processor、UI 投影与 reducer 复用

上一节讲到 OpenAI agent processor 的状态模型：输入事件进入历史，系统提示和模型名来自状态，响应流再被翻译回事件。到了 00:45:02，现场演示没有按这个理想路径继续推进。讲者明确承认 OpenAI agent demo 跑不起来，甚至遇到了 quota 与示例仓库状态不一致的问题；他判断如果继续现场修代码，大概要额外花十分钟集中排查。这个失败很有信息量：LLM agent 版本的现场实现不稳定，但底层事件流、ping-pong processor、UI feed 和 circuit breaker 这些更小的构件已经能说明架构真实长相。

这一段不能被写成“OpenAI agent demo 已经顺利跑通”。视频里的事实更直接：OpenAI agent demo 没有现场成功，ping-pong 简例可验证，后续问答把真正运行在服务内部的 processor 暴露出来。架构判断应建立在这些可观察构件上。

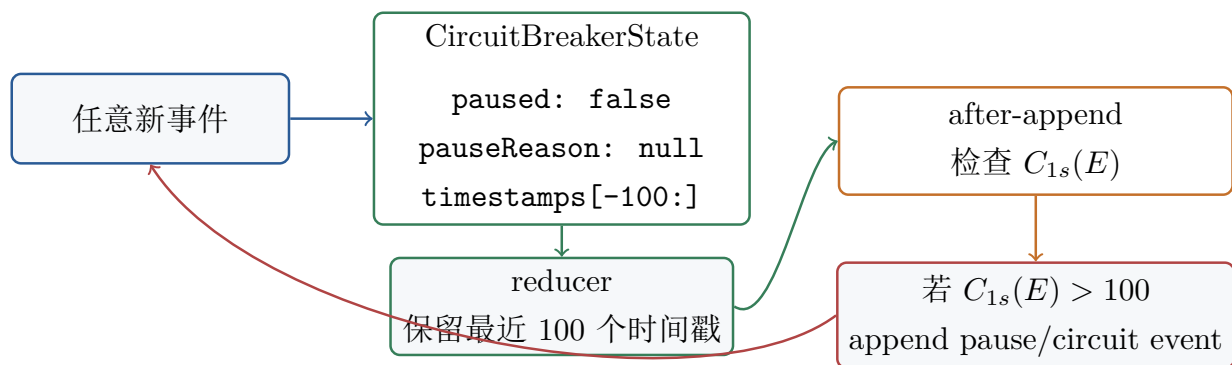
6.1 失败现场给出的有效信息

讲者先试图退回到最简单的 ping-pong processor：看到 ping，追加 pong。这个例子很小，却能验证三条基础链路：

- 事件可以被追加到 stream，服务端给出 offset 和 created_at。
- processor 可以观察事件，并按事件类型决定是否行动。
- processor 的行动仍然表现为追加新事件，结果不会藏在本地内存或外部日志里。

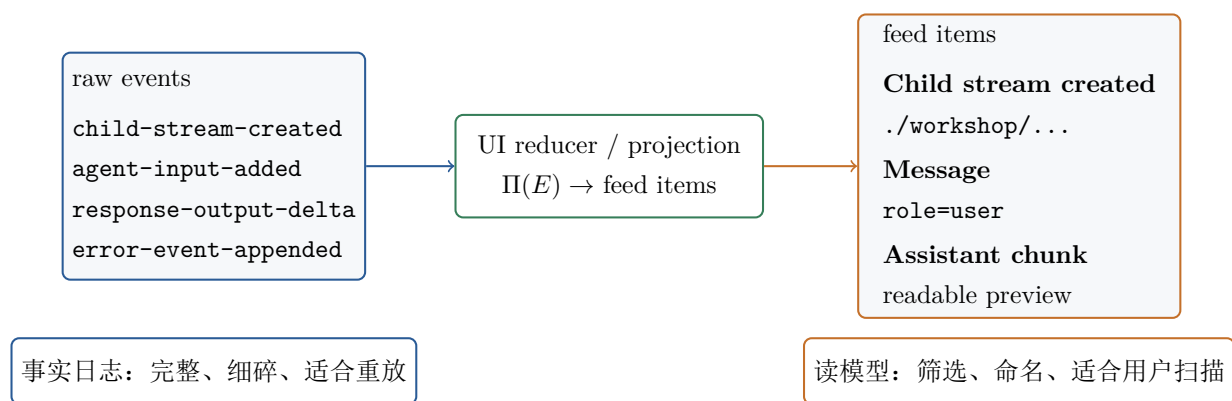
随后有观众追问 stream processor 到底消费什么、物化什么。讲者的回答把抽象压到了一个很窄的接口：系统里发生的每件事，包括 OpenAI streaming API 返回的 chunk，都可以落成事件。所谓 chunk，可以理解为“模型这次流式输出吐出的一小段文本或结构化增量”。如果响应是 $\langle c_1, c_2, \dots, c_m \rangle$ ，每个 c_i 都能成为一个事件 e_i ，例如 `response.output_text.delta` 或同类业务事件。这样一来，UI、熔断器、agent 历史、调试器看到的是同一种材料：按 offset 排列的事件序列 $E = \langle e_1, e_2, \dots, e_n \rangle$ 。

失败现场暴露出的核心架构约束是：事件流不只是 LLM 输入输出的日志，还是系统内部 processor 的共同接口。OpenAI agent demo 没有跑通，不影响这个约束本身；因为 ping-pong、circuit breaker 和 UI feed 都已经使用同一套“事件进入 reducer，必要时追加事件”的机制。



这里的熔断器也是 **processor**：用同一套 **reducer/after-append** 模式保护事件流。

图 12: circuit breaker processor 的内部机制：reducer 累积最近事件时间戳，after-append 在 1 秒窗口内发现事件过量时追加暂停类事件。¹²



projection 像把流水账整理成对账单：来源仍是事件，呈现层换成更容易看的条目。

图 13: UI feed items 是从 raw events 派生的投影：界面 reducer 选择有意义的事件，转换成更适合阅读的 feed item，而原始事件日志仍保留完整事实。¹³

6.2 内置 circuit breaker processor：用事件保护事件流

前面章节已经提过 **pause/resume** 是用户可见的流控制事件。本节关注它背后的内部实现。讲者在 00:47:50 左右说明，服务内部有一个真实运行的 **熔断器处理器**（**circuit breaker processor**）。它的初始状态至少包含三个字段：

```

1 type CircuitBreakerState = {
2   paused: boolean;           // 初始值 false
3   pauseReason: string | null; // 初始值 null
4   recentTimestamps: number[]; // 最近事件的 created_at 时间戳
5 };
  
```

Listing 8: circuit breaker state 的最小形状

这里的 **paused=false** 表示当前 stream 还能接受新事件；**pauseReason=null** 表示尚无暂停

原因：`recentTimestamps` 保存最近事件的时间戳，用于判断事件速率。reducer 的工作很克制：每来一个事件，只把它的 timestamp 放进列表，并保留最近约 100 个 timestamp。after-append 再检查这些时间戳是否过密。一旦最近 100 个事件没有跨过 1 秒窗口，系统就追加一个暂停类事件，让 stream 进入 paused 状态。

用符号写，令 $C_{\Delta}(E)$ 表示事件序列 E 在时间窗口 Δ 内的事件数量：

$$C_{\Delta}(E) = |\{e_i \in E \mid \text{created_at}(e_i) \in [t - \Delta, t]\}|$$

本视频中的速率保护可以概括为：

$$C_{1s}(E) > 100 \Rightarrow \text{append}(\text{paused})$$

它的意思是：如果最近 1 秒内的事件数量超过约 100 个，after-append 就追加一个表示暂停的事件。这个暂停事件进入同一条 stream，随后普通的 reducer、UI 和审计工具都能看见它。这里的 $\Rightarrow$ 表示工程规则里的触发关系；左边是速率条件，右边是系统采取的保护动作。

circuit breaker 这个词来自电路断路器。电流过载时，断路器先切断电路，保护设备；事件流过载时，熔断器追加 paused 事件，保护 stream 免受无限循环、插件互相打架、错误重试风暴的冲击。它保护的是整条 append-only 日志的可用性，而非某个单独函数。

这个例子也说明 reducer 与 after-append 的边界。reducer 只做可重放的状态计算：

$$S_{t+1} = r(S_t, e_{t+1})$$

其中 S_t 包含 `paused`、`pauseReason` 和最近 timestamp 列表。真正改变系统行为的动作是 after-append 里的 `append(paused)`。这样设计有两个好处：第一，重放历史时能稳定重建“当时是否应当暂停”；第二，暂停本身也是事件，可以被 UI 展示、被审计系统检查、被后续 resume 事件解除。

6.3 正式定义：projection 与 read model

问答继续推进到 UI。讲者指出，网页里那些排版更友好的 feed item 来自 raw event 的派生，属于 UI 对象。这就引出本节需要正式定义的两个术语。

投影（projection） 是从事件日志派生读侧数据的过程或函数。给定事件序列 $E = \langle e_1, e_2, \dots, e_n \rangle$ ，投影函数 Π 产生一个面向读取、展示或查询的结构：

$$R = \Pi(E)$$

其中 R 就是 **读模型（read model）**：它是为了高效读取、筛选、排序、渲染而生成的派生视图，事实来源仍然是事件日志。直观类比是账本与报表：账本记录每一笔原始流水，报表按月

份、类别、项目汇总流水。报表可以重算，账本才是事实来源。放到事件溯源里，raw events 是账本，feed items 是报表。

UI feed 的 processor 可以写成类似下面的归约：

```
1 type FeedState = {
2   items: FeedItem[];
3 };
4
5 function feedReducer(state: FeedState, event: Event): FeedState {
6   if (!isInterestingForFeed(event)) return state;
7
8   return {
9     items: [
10      ...state.items,
11      {
12        id: `${event.streamPath}:${event.offset}`,
13        title: renderTitle(event),
14        body: renderBody(event),
15        timestamp: event.created_at,
16        sourceEventOffset: event.offset,
17      },
18    ],
19  };
20 }
```

Listing 9: UI feed items 从 raw events 派生

这个 read model 有意携带 `sourceEventOffset`。原因很实际：用户点击 UI 上的一条 feed item 时，系统最好能回到原始 event，查看完整 payload、事件类型、stream path、created_at 和前后上下文。没有这个反向链接，漂亮 UI 会变成丢失证据链的截图。

feed item 是 projection/read model，来源于 raw event。raw event 负责保留事实，feed item 负责让人更快看懂事实。二者的生命周期也不同：raw event 应按 append-only 规则保存；feed item 可以在前端、缓存或服务端索引里重建。

6.4 UI 也是 processor：读侧同样在 reducing

讲者说“这个 UI 也是一个 stream processor”，意思很具体：UI 观察同一条事件流，对它认为有意义的事件运行 reducer，然后产出 feed items。它不会把每个 raw event 原样展示给人，因为 raw event 的 payload 更像机器接口；UI 需要把 agent input added、response chunk、paused、invalid event appended 等事件投影成可读条目。

这里容易踩的坑是把 projection 当成“前端格式化”。更准确地说，projection 是读侧建模。它决定哪些事件进入视图、事件之间如何合并、错误如何高亮、流式 chunk 如何拼接、暂停原因如何呈现。比如 OpenAI streaming chunks 都作为事件进入 stream 后，UI 可以选择逐条显示

delta，也可以把相邻 delta 合并成一条正在增长的 assistant message。两种 UI 行为面对同一组 raw events，区别只在投影函数 P 。

这种设计带来一个重要工程收益：UI 崩了、刷新了、换前端框架了，都可以重新读取 E ，再算出 $R = \Pi(E)$ 。在调试 agent 时，这比只保存最终 DOM 或浏览器状态可靠得多。事件像素材胶片，read model 像剪辑版；剪辑可以重做，素材必须保真。

projection 不能偷偷承担写侧事实。比如 feed item 标记“用户已批准工具调用”，这个批准必须有对应 raw event；只把批准状态塞进 UI read model，会让后端 processor、审计系统和其他客户端看不见关键事实。

6.5 reducer 可导入复用：共享事件类型上的抽象叠加

这一段问答的最后一个重点是 reducer 复用。讲者举了 Pi agent harness processor 的例子：某个 agent harness 已经定义了一组事件类型，也提供了 reducer。别的 processor 可以直接 import 这个 reducer，在自己的状态计算里运行它，再在其上叠加更高层抽象。

形式化地说，假设已有 reducer：

$$r_{\text{base}}(S, e) \rightarrow S'$$

新 processor 可以把它包成组合 reducer：

$$r_{\text{app}}((S_{\text{base}}, S_{\text{extra}}), e) = (r_{\text{base}}(S_{\text{base}}, e), r_{\text{extra}}(S_{\text{extra}}, e, S_{\text{base}}))$$

这里 S_{base} 是别人定义的 agent 状态，比如 history、tool call、model config； S_{extra} 是本 processor 自己新增的状态，比如安全评分、UI 分组、指标统计、RAG 命中记录。因为 reducer 是同步函数，导入并运行它的成本接近普通函数调用。它不需要远程 RPC，也不需要让原作者暴露一整套插件 API。

这个复用模式的边界也要说清楚。共享 reducer 的前提是共享事件类型语义：agent input added 到底表示用户输入已经接受、正在等待模型、还是仅仅进入草稿区，必须写清楚。事件类型像公共语言里的词；词义不稳，所有依赖它的 reducer 都会产生误判。工程上至少需要三样东西：

- 事件 type 的稳定命名与文档，最好能链接到 schema。
- payload 的结构约束，例如用 Zod、JSON Schema 或 OpenAPI 描述字段。
- reducer 的版本策略，避免新旧 processor 对同一事件序列算出互相矛盾的状态。

这也解释了为什么讲者反复强调“reducers can run in many different places”。同一批 raw events 可以在服务内部运行 circuit breaker reducer，在 UI 里运行 feed reducer，在 agent harness 里运行 history reducer，在第三方扩展里运行安全或指标 reducer。它们各自维护不同状态，却共享同一条事实链。

6.6 失败 demo 的工程含义

从演示效果看，这一段确实狼狈：OpenAI agent processor 没跑起来，讲者最后甚至承诺之后录一个更完整的视频。但从架构角度看，这段反而把系统的关键层次暴露得更清楚：

1. 最小可验证层：ping-pong processor 证明事件观察和追加闭环成立。
2. 服务内置层：circuit breaker processor 证明平台内部也用 reducer/after-append 管理保护逻辑。
3. 读侧投影层：UI feed processor 证明 raw events 可以派生成人类可读的 read model。
4. 组合复用层：第三方 processor 可以导入 reducer，围绕共享事件类型继续叠加行为。

这四层比一次顺滑的 LLM demo 更有诊断价值。顺滑 demo 只会显示“模型能回复”；失败后的问答显示“系统如何在模型、UI、保护机制和第三方扩展之间共享同一种事件材料”。对于 agent harness，这才是能不能长期演化的关键。

6.7 本章小结

本节覆盖的是 00:45:02 到 00:50:36 的失败现场与问答。OpenAI agent demo 没有跑通，ping-pong 简例仍可验证事件驱动闭环；观众提问引出所有 streaming chunks 都能作为 events，reducers 可以在多个位置运行。服务内部的 circuit breaker state 至少包含 `paused`、`pauseReason` 和最近 timestamp 列表，约 100 events/second 会触发 $C_{1s}(E) > 100 \Rightarrow \text{append}(\text{paused})$ 这样的暂停动作。

同时，UI feed items 是从 raw events 派生出的 projection/read model，负责展示和查询，不承担事实来源职责。reducer 因为是同步函数，可以被其他 processor 导入复用；围绕共享事件类型，系统能够把 agent harness、UI、熔断器和第三方扩展叠加在同一条事件流上。

7 动态 Worker：把处理器代码打包成事件

上一节已经把现场 demo 的失败转成了有用的架构信息：系统内部的熔断器、UI feed 投影、ping-pong 示例都可以写成 stream processor。00:50:37 之后，讲者把这套思路推到更激进的一步：处理器代码本身也可以作为事件追加到 stream 里。这样，一条原本只存事实的流，会在追加某个配置事件之后获得新的运行行为。

7.1 dynamic worker 的正式定义

动态工作器 (dynamic worker) 是由事件流中的配置事件启动或更新的处理器运行环境。它读取一条 stream 里的配置事件，例如 `dynamic worker configured`，从 payload 中取出处理器代码、环境映射和运行参数，然后在平台侧创建可执行的 worker，用同一条 stream 的后续事件驱动它。

更形式化地说，若事件流为

$$E = \langle e_1, e_2, \dots, e_n \rangle,$$

其中某个配置事件

$$e_k = (\tau = \text{dynamic worker configured}, p = \{\text{script}, \text{env}, \dots\})$$

定义了处理器 P ，那么 k 之后的事件可以被该处理器观察并转换为新事件：

$$P(E_{\geq k}) \rightarrow E'$$

这里的 E' 会继续追加回 stream，成为可订阅、可审计的事实，例如 `pong`、`worker error`、`compile error` 或 `agent response chunk`。直观类比是：普通事件像账本流水，`dynamic worker configured` 像把一套“自动记账规则”贴到这本账本上；之后每来一笔流水，这套规则都会被触发。

dynamic worker 的核心判断 dynamic worker 的关键点是“代码以事件形式配置运行环境”。流服务不只保存 `ping`、用户输入和模型输出，也保存“从现在起怎样处理这些事件”的程序事实。这个设计让部署动作变成 `append` 动作，同时也把动态代码执行、权限、依赖和 `secret` 管理推成必须正视的工程问题。

7.2 stream 隐式创建：先有路径，再长出行为

讲者在 00:50:38 先强调了一件很朴素的事：所有 stream 都可以隐式创建。开发者向 streams API 下的任意 path 发起 POST，服务端就会为这个 path 接受第一条事件。这里没有单独的“创建数据库表”步骤，也没有预注册 agent 的仪式。路径本身就是命名空间，第一条事件就是这个命名空间的起点。

这使 dynamic worker 的体验非常直接：

1. 选择一个新路径，例如 `/workshop/alice/dynamic-ping-pong`。
2. 向这条路径追加 `dynamic worker configured`。
3. 再追加一条 `ping`。
4. worker 观察到 `ping`，追加 `pong`。

在这个流程里，流的“存在”和“有行为”是两层概念。第一次 POST 让 path 成为 stream；配置事件让 stream 关联一个处理器；后续业务事件触发处理器产生输出。用系统语言说，stream 是持久化日志，dynamic worker 是依附在日志上的执行投影。

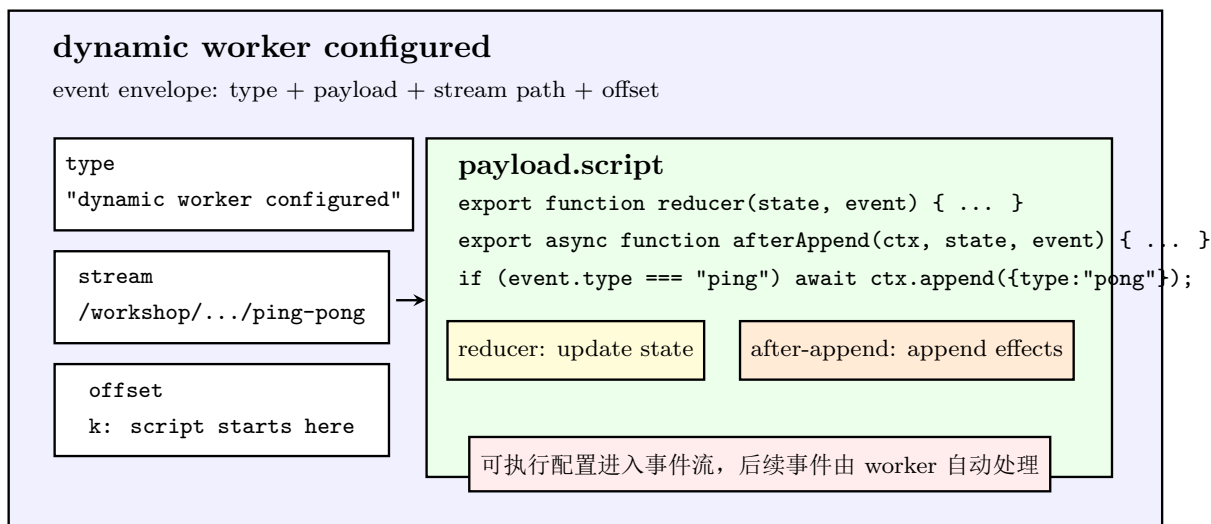


图 14: dynamic worker configured 把处理器代码塞进事件 payload, script 字符串同时携带 reducer 与 after-append。配置事件的 offset 变成行为生效的时间坐标。¹⁴

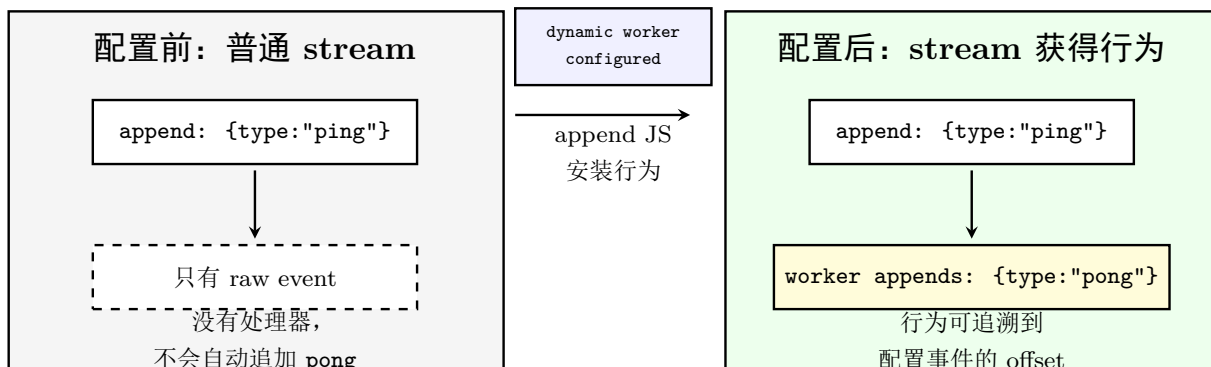


图 15: append 一段 JavaScript 之后，原本只保存事实的 stream 可以响应 ping 并追加 pong。这里的变化是“流获得处理器行为”。¹⁵

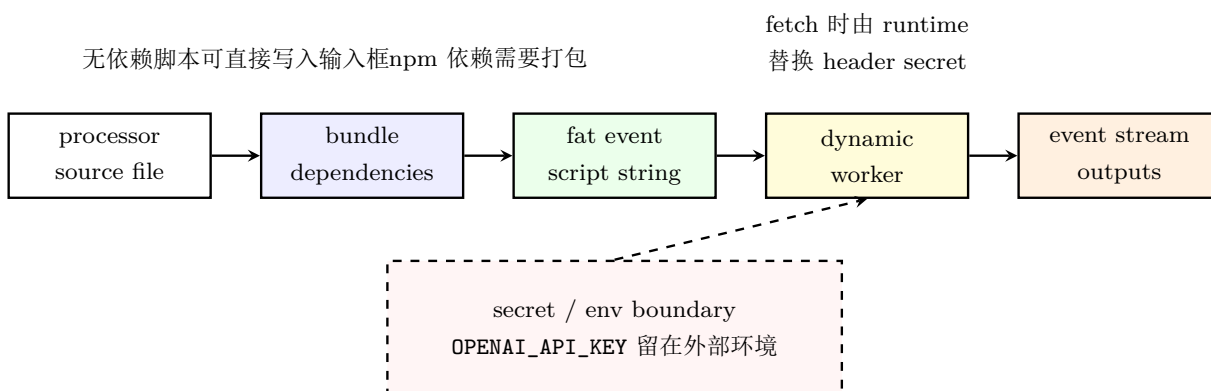


图 16: 从源码到动态 worker 的安全边界：依赖可以被 bundle 进 fat event，API key 这类 secret 应通过外部 env 注入，避免进入 append-only 事件日志。¹⁶

7.3 configured payload: script 字符串里的 reducer 与 after-append

dynamic worker configured 的 payload 中最重要的字段是 script。讲者明确说，这个 script 是一个字符串，字符串内部包含 reducer 和 after-append hook。一个简化的事件形状可以写成：

```
1 {  
2   "type": "dynamic_worker_configured",  
3   "payload": {  
4     "script": "export function reducer(state, event) { ... } \nexport async function  
5     afterAppend(ctx, state, event) { ... }"  
6   }  
}
```

Listing 10: dynamic worker configured 的简化 payload

这里很容易踩到一个认知坑：script 属于可执行配置，不能当成说明文字。worker runtime 会把它解释成 stream processor：reducer 仍然负责同步归约状态，after-append 仍然负责副作用。前面章节定义过的边界没有变化，只是 processor 的交付方式从本地文件、服务器代码，变成了事件 payload 里的字符串。

最小 ping-pong 版本可以想成下面这种结构：

```
1 export function reducer(state = {}, event) {  
2   if (event.type !== "ping") return { shouldPong: false };  
3   return { shouldPong: true, pingOffset: event.offset };  
4 }  
5  
6 export async function afterAppend(ctx, state, event) {  
7   if (!state.shouldPong) return;  
8  
9   await ctx.append({  
10     type: "pong",  
11     payload: { inReplyTo: state.pingOffset }  
12   });  
13 }
```

Listing 11: script 字符串内部的最小处理器轮廓

当这段代码被打进 dynamic worker configured，append 后的 stream 就会对 ping 产生 pong。这件事的工程含义很具体：行为的安装、行为的更新、行为的审计，都可以回到 event stream 里查。某条 stream 为什么突然开始回复 pong，答案可以追到某个 offset 上的配置事件。

代码作为事件的尺度 把代码放进事件，类似把数据库触发器、Serverless 函数配置和审计日志压到同一个时间轴上。差别在于，event stream 的 offset 给了它明确坐标：从 offset k 开始，script 生效；从 offset m 开始，新 script 覆盖旧行为。调试时可以问一个非常具体的问题：某个输出事件发生时，生效的是哪一次配置？

7.4 AI agent 版本: processor file bundle 成 fat event

ping-pong 只是证明 dynamic worker 真的能让流获得行为。讲者接着提到 AI agent 版本: 如果本地有一个 processor file, 里面用 `defineProcessor` 导出处理器, 那么可以把这个文件 bundle 成一个事件 payload。这个 payload 不只包含手写的几十行 agent 逻辑, 还包含运行所需的编译产物, 因此讲者称它接近一个 **fat event**: 事件本身携带足够多的代码, 让平台可以直接启动 worker。

这条路径可以分成四步:

1. 开发者在本地写 `agent.processor.ts`, 其中包含 reducer、after-append、事件类型匹配和 LLM 调用逻辑。
2. bundler 把 processor file 编译成浏览器或 worker 可执行的 JavaScript。
3. 编译结果被放入 dynamic worker configured 的 `script` 字段, 作为 fat event 追加到目标 stream。
4. 后续 `agent input added` 事件进入 stream 后, dynamic worker 归约 history, 触发 OpenAI Responses API 请求, 并把输出 chunk 或完成事件追加回来。

这解释了讲者说的“40 行基础 AI agent 可以 append 到任意 stream, 然后这条 stream 就变成 AI agent”。这个说法有实验性质, 现场实现也不稳定; 可验证的核心仍然清楚: 处理器代码可以 bundle 到事件里, 事件可以配置 worker, worker 可以把输入事件变成响应事件。

7.5 secret 不能进 event: env 与 secret 注入

dynamic worker 最危险的诱惑, 是把所有东西都塞进 `script`。讲者在 00:52:16 专门指出, OpenAI API key 不能放在事件中。原因很直接: 事件流是审计日志, 可能被 UI、SSE 订阅、调试工具、第三方 processor、备份系统读取; 一旦 secret 进入事件, 就等于把密钥写进了账本, 后续只能轮换密钥, 无法真正抹除传播痕迹。

正确做法是把密钥放在流外侧的 env/secret 存储里。worker 发起 `fetch` 时, 由运行环境把占位符替换成真实 header。例如:

```
1 await fetch("https://api.openai.com/v1/responses", {
2   method: "POST",
3   headers: {
4     "Authorization": `Bearer_${ctx.secrets.OPENAI_API_KEY}`,
5     "Content-Type": "application/json"
6   },
7   body: JSON.stringify(requestBody)
8 });
```

Listing 12: 概念示例: secret 由运行环境注入

这里的 `ctx.secrets.OPENAI_API_KEY` 代表运行环境提供的 secret 引用,不代表事件 payload 中真的有密钥字符串。更稳妥的设计还应记录“使用了哪个 secret 名称、由哪个 worker 版本请求、请求对应哪个 offset”,但不要记录 secret 值本身。

event stream 不能存 secret OpenAI API key、数据库密码、Webhook 签名密钥、OAuth refresh token 都不能进入事件 payload。事件流的优势是可复制、可订阅、可审计;这些优势会把 secret 泄漏放大。密钥应通过 env、secret manager 或平台绑定注入,事件中最多保存 secret 名称、版本号和授权范围。

7.6 npm 依赖需要 bundle: 输入框适合小脚本, 工程代码需要打包

讲者还指出, UI 输入框里直接写 JavaScript 时没有依赖环境。也就是说, script 可以写标准 JavaScript 和平台提供的上下文对象,但不能随手 import 任意 npm 包。若处理器需要 zod、OpenAI SDK、schema matcher、tokenizer 或内部工具库,就必须先 bundle。

这个约束背后的原因很朴素: worker runtime 收到的是一个字符串。它在执行时不能临时猜测依赖版本、联网安装包、解析本地路径。依赖图必须在追加事件前被压平。工程上常见的 bundle 产物包含:

- 入口 processor 的代码。
- 被 import 的 npm 包或本地模块。
- 必要的 polyfill 或 runtime helper。
- 对 secret、环境变量、外部 API base URL 的占位引用。

讲者设想过一个更进一步的 unbundled worker 事件: payload 里放 `package.json` 和 script, 由另一个 processor 负责安装、编译、打包,再生成 fat event。这个构想把 event stream 推成一种轻量 IDE: agent 可以追加“请把自己改成另一种功能”的代码事件,构建 processor 再把可运行结果追加回来。这个方向很强,也很危险;权限、供应链、构建隔离和可回滚策略都要在 Section 8 的安全边界里继续收束。

7.7 错误也事件化: 编译失败、运行失败、修复建议

如果 script 写错会发生什么? 讲者的回答很符合全片主线: 任何错误都应该变成事件。当前 demo 是 JavaScript evaluator, 因此没有 TypeScript 类型错误; 如果加入 TypeScript 编译步骤, 就可以在编译失败时追加 `typescript compiler error`。如果运行时抛异常, 就追加 `dynamic worker error`。如果再接一个 LLM 修复器, 它可以读取错误事件和原始代码, 追加 `dynamic worker fix suggested` 或新的 `dynamic worker configured`。

这条错误链可以写成:

configured → compile error → fix suggested → configured

或者运行期链路：

ping → worker error → diagnostic event

这里的重点是错误不会消失在平台日志里。平台日志可以辅助排查，但 agent harness 的事实层应能回答：哪段代码失败、处理哪条事件失败、失败时 state 大致是什么、系统是否生成过修复建议。错误事件化让动态代码具备审计入口，也让自动修复循环有了输入材料。

7.8 三种运行路径：本地 SSE、Web service、Cloudflare dynamic worker

讲者最后把 processor 的部署路径讲成三个层级。

第一种是本地 SSE 路径。开发者在自己的电脑上打开 `?live=true` 或 SDK 的 `stream(live: true)`，本地进程持续接收事件，再运行 reducer 和 after-append。它最适合开发、调试和观察，因为代码改动快，日志就在终端里。代价也清楚：电脑关机、网络断开、进程崩溃时，processor 不会实时处理新事件，只能在重启后 catch-up。

第二种是 Web service 路径。开发者把 processor 部署成普通服务，例如 Cloudflare Workers、Vercel、Fly.io 或自家服务器，然后让 stream 服务通过 push subscription 通知它“有新事件需要处理”。这种方式更接近传统生产部署：服务有自己的版本、监控、灰度和告警；stream 负责投递事件，processor 负责处理并 append 结果。

第三种是 Cloudflare dynamic worker 路径。在视频中的具体实现里，dynamic worker configured 会让平台启动一个小型 Cloudflare dynamic worker。开发者不再单独部署服务，append 配置事件就完成部署。它的吸引力很明显：把 agent 行为发布成事件，任意 stream 都可以获得同一段处理逻辑。它的风险同样明显：平台需要限制 CPU、内存、网络访问、secret 范围、依赖来源和循环写入。

这三条路径可以看成同一段 processor 的成熟度阶梯：先在本地 SSE 跑通，再部署成 Web service，最后把足够小、足够受控的处理器交给 dynamic worker。对 AI agent 来说，这个阶梯很实用：开发时看得到每个 offset，生产时有服务边界，高频复用时再考虑事件化部署。

7.9 本章小结

本节解释了 dynamic worker 的核心机制：stream 可以通过任意 path 的 POST 隐式创建；dynamic worker configured 的 payload 中有 script 字符串；script 内包含 reducer 和 after-append；追加配置事件后，原本没有行为的 stream 可以在收到 ping 后产生 pong。

更复杂的 AI agent 版本可以把 processor file bundle 成 fat event，让一条 stream 获得 LLM agent 行为。工程边界也很硬：OpenAI API key 和其他 secret 不能写入 event，需要由 env/secret 注入；npm 依赖不能靠运行时临时安装，需要预先 bundle；编译错误、运行错误和修复建议都应继续事件化。运行路径有三种：本地 SSE 适合调试，Web service 适合常规生产部署，Cloudflare dynamic worker 适合把受控处理器直接打包成事件来发布。

8 总结与延伸：分布式插件、安全检查、最终一致性与权限边界

本章收束 00:56:27–01:04:05 的讨论。讲者在结尾处没有把这个系统包装成已经成熟的产品形态，反而反复承认现场演示仍然粗糙、解释也有些吃力；但他真正想留下的是一个架构判断：如果 agent 的行为都落在 durable stream 上，插件就不必被塞进同一个进程、同一个 SDK、同一组私有 hook 里。安全检查、RAG 补充、审计、UI 投影、动态 worker，都可以作为独立 processor 观察同一条事件流，并在时机合适时追加新事件。

这也是全片的最后一层抽象：event stream 既是事实账本，也是分布式协作面。前面几章已经讲过事件信封、offset、reducer、after-append、LLM request、UI projection 和 dynamic worker；本章把这些线索放回工程现实中，讨论性能、成本、安全、权限和最终一致性。换句话说，一个真正可用的事件溯源 agent harness，既要能让第三方服务快速插入上下文，也要能在第三方服务来不及响应时继续前进；既要开放 public namespace 方便实验，也要给每个事件留下来源证明和权限边界。

8.1 第三方安全 checker 的 200ms 窗口

讲者举的例子很具体：一个 prompt-injection protection 服务可以运行在另一台机器上。它不必成为 agent 进程内的库，也不必通过 MCP 工具等待 agent 主动调用。更合理的接口是：它订阅 agent stream，看到 agent 即将发起新的 LLM request，便在一个很短的时间窗口内判断是否需要追加安全上下文、警告或拦截建议。

这个等待窗口可以记为

$$w = 200\text{ms}.$$

w 表示 agent loop 在发起下一次 LLM 请求前愿意留给外部 processor 的最长等待时间。这里的 200ms 更像延迟预算，带有工程经验色彩：足够让局域网、边缘节点或较快的云服务返回一个小型判断；也短到不会把交互式 agent 变成明显卡顿的系统。直观类比是电梯门关闭前的短暂停顿：有人刚好赶上，就进入本轮；没有赶上，电梯继续运行，下一层还有机会。

第三方 safety checker 的关键价值在于“就近介入”，目标并非“接管主循环”。agent loop 最多等待约 $w = 200\text{ms}$ ，checker 如果及时追加上下文，LLM request 就带上这段上下文；checker 如果没有及时返回，主流程仍然发出请求。这个设计把安全服务从同步阻塞点降级为限时协作者。

用事件流语言表达，设原始事件序列为 E ，第 j 个 processor 为 P_j ，它观察事件流后可能追加新事件：

$$P_j(E) \rightarrow E'.$$

这里 E' 指包含追加事件后的新序列，历史 E 依然保持原样。比如安全 processor 看见 `llm.request.pending` 事件后，追加 `safety.context.added`；agent loop 在 w 内看见这条补充事件，就把它纳入本轮请求。若 P_j 没赶上， E 仍继续向前，稍后的 `safety.warning` 仍可用于审计、后续纠偏或暂停下一轮。

这一点解释了讲者为什么认为 stream 应该能“主动触达 processor”。如果只能靠 agent 主动调用某个 MCP 工具，第三方服务就很难在“模型请求即将发生”这个瞬间介入。事件流让另一台机器上的 checker 可以被新事件唤醒，或者通过 SSE/push subscription 近实时观察到新事件。插件商业化也由此变得自然：安全服务提供商只需承诺自己的 processor 能理解事件协议、在延迟预算内追加结果，并按调用量或订阅量计费。

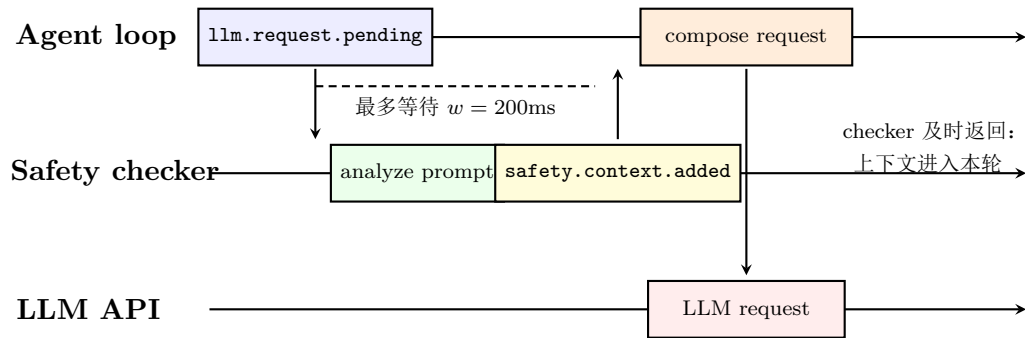


图 17: safety checker 的 just-in-time 窗口: agent loop 在发起 LLM 请求前最多等待约 200ms, 及时到达的检查结果被纳入本轮上下文。¹⁷

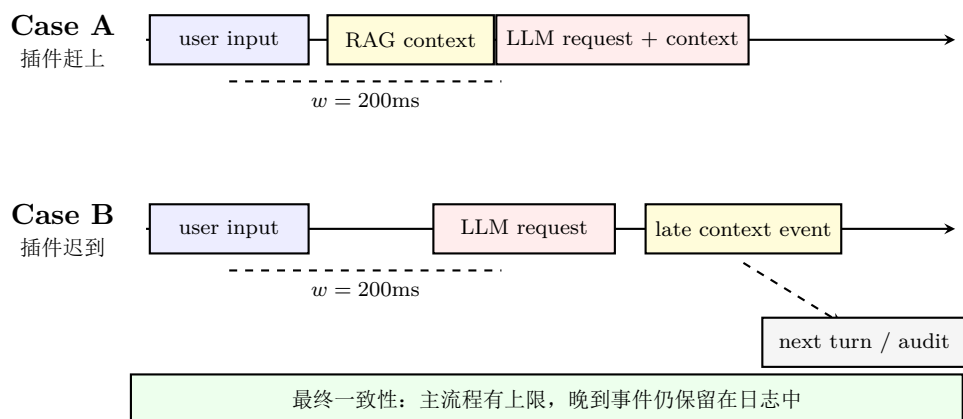


图 18: eventual consistency 的两条时间线: 外部 processor 在窗口内赶上就补上下文, 超出窗口则主流程继续, 晚到结果进入后续轮次或审计。¹⁸

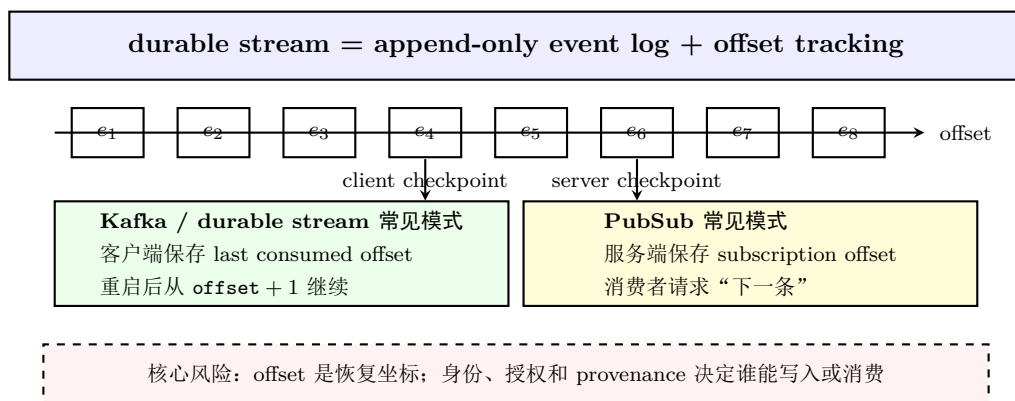


图 19: durable stream、Kafka 与 PubSub 的 offset tracking 对比: 共同基础是只追加日志, 差别在消费进度由客户端还是服务端保存。¹⁹

8.2 为什么第三方 before hook 风险很高

讲者在结尾处明确反对第三方 before hook。before hook 指事件进入 stream 之前、LLM request 发出之前或框架执行某一步之前，由第三方代码获得同步拦截权。它看起来很有吸引力：只要 checker 没通过，就让事件无法进入系统；只要 RAG 没返回，就暂停模型请求。但这种设计把第三方插件放到了主路径的闸门位置，性能、成本和缓存都会被放大影响。

第一类问题是性能。交互式 agent 的体验由尾延迟决定。只要一个 before hook 偶发变慢，整轮 LLM request 就被拖住；多个 before hook 串联时，总延迟近似累加：

$$T_{\text{request}} \approx T_{\text{base}} + \sum_{j=1}^m T_{P_j}.$$

其中 T_{base} 是 agent 原本准备请求的耗时， T_{P_j} 是第 j 个插件的同步等待时间。插件数量越多，最慢插件越容易支配用户感知。对于代码 agent、终端 agent、聊天 agent，这种阻塞会很快变成产品层面的退化。

第二类问题是成本。before hook 往往会诱导插件把更多上下文、更多检查结果和更多中间状态塞进请求前路径。每次请求都等待、拼接、改写上下文，token 数量和外部服务调用次数都会上升。更麻烦的是，很多 hook 的返回内容具有短时随机性或请求相关性，容易让原本可复用的 prompt 前缀发生变化。

第三类问题是 context caching。context caching 指模型服务对稳定的长前缀上下文做缓存，让后续请求少付延迟和成本。若 before hook 在系统提示、工具说明或历史上下文前部频繁插入变化内容，缓存键会不断失效。缓存可以类比图书馆把常用资料提前放在桌上：如果每次开会前都把第一页、目录和装订顺序改掉，工作人员就很难复用上一轮准备好的材料。

第三方 before hook 的危险不只在“代码会出 bug”。更现实的风险是它把外部服务放进主路径，直接放大尾延迟、token 成本和 context caching 失效率。对 agent loop 来说，主路径越长，越难判断到底是模型慢、网络慢、checker 慢，还是某个插件把上下文缓存打碎了。

这并不意味着所有前置检查都该消失。讲者也承认，系统内部的 built-in processor 可能拥有更强权限，甚至可以阻止某些事件被追加。关键边界在“第三方”。平台自己的安全边界、schema 校验、认证授权、速率限制可以位于写入前；第三方插件更适合在写入后观察事件，并在短窗口内追加建议、上下文或告警。

8.3 eventual consistency: 赶上就补，赶不上继续

Eventual consistency (最终一致性) 是分布式系统中的一致性模型：多个组件短时间内可能看见不同状态，但只要没有新的冲突写入，系统会通过事件传播、重试、补偿和重放逐步收敛到同一个可解释状态。在本章语境中，它强调主流程和外部 processor 之间采用有限等待、异步补充、后续收敛的协作方式，含义远比“随便晚点也行”严格。

最贴近视例的例子是 RAG processor。它可能维护一个 Notion 知识库索引，持续观察 agent stream。当用户输入出现后，RAG processor 判断是否有相关资料，并尝试追加一段

`rag.context.added` 事件。若它在 $w = 200\text{ms}$ 内赶上，本轮 LLM request 就带上这段资料；若它没赶上，agent 依然请求模型，RAG 结果可以进入下一轮、触发修正、补充引用，或者只作为审计线索存在。

最终一致性可以类比多人协作文档里的评论系统：正文编辑者不会因为某个审阅者网络慢就永远停笔；审阅意见赶上当前段落，就被立刻采纳；晚到的意见仍然留在记录里，之后可以修订。事件流中的 processor 也是如此：它们围绕同一条事实流协作，短暂不同步由 offset、时间窗口和补偿事件来吸收。

形式化地看，设 agent 主循环生成请求事件 e_q ，外部 processor 集合为

$$\mathcal{P} = \{P_1, P_2, \dots, P_m\}.$$

每个 P_j 都观察 E ，并可能追加 ΔE_j 。agent loop 在窗口 w 内收集到的补充集合可写为

$$C_w(e_q) = \{\Delta e \mid \Delta e \text{ 在 } e_q \text{ 后 } w \text{ 内到达且与本轮请求相关}\}.$$

随后请求上下文近似为

$$H' = H \cup C_w(e_q).$$

若某个补充事件晚于 w ，它不进入 H' ，但仍成为事件流的一部分，供后续 processor、审计 UI 或下一轮请求读取。这个模型的好处是主流程有清晰上限，第三方服务也有明确承诺：能在窗口内提供价值最好，超时也不会拖垮系统。

这正是讲者用“distributed systems things”概括的工程含义。系统承认网络延迟、进程调度、远端服务冷启动和索引查询都会发生波动，于是把同步拦截改成异步追加，把绝对前置改成限时机会，把“所有插件必须完成”改成“赶得上的插件参与本轮”。

8.4 durable stream: append-only event log 加 offset tracking

Durable stream (持久流) 在本章中有一个很朴素的定义：它是 **append-only event log** 加 **offset tracking**。append-only event log 表示事件只能追加，历史事件按顺序保留；offset tracking 表示消费者能够知道自己处理到哪里，重启后从正确位置继续。两者合在一起，就把“实时消息”提升成“可恢复的事实序列”。

可以写成：

$$E = \langle e_1, e_2, \dots, e_n \rangle, \quad \text{offset}(e_i) = i.$$

某个消费者 c 的进度为 o_c 。当它处理到 e_k 后，进度更新为

$$o_c \leftarrow k.$$

如果 c 崩溃后重启，它不需要猜测自己错过了什么，只需从 $o_c + 1$ 继续读取。这个机制与前文的 processor catch-up 对上了：reducer 可以重放旧事件恢复状态，after-append 阶段再按运行时策略决定哪些副作用仍应执行。

讲者提到 durable streams API specification，也提到一些类似 JetStream 的系统。这里无需绑定某个具体厂商。重点是形状：一边像 queue，能把工作交给消费者；一边像 pub/sub，能把新事件推给订阅者；一边又像 streaming database，能保存历史、跟踪 offset、支持重放和恢复。对于 agent harness，这个形状比普通 webhook 更强，因为 webhook 常常只负责“现在投递一次”，而 durable stream 还能回答“谁看过了、看到哪了、能否补看”。

Durable stream 的核心落在持久保存、按 offset 重放、让每个 processor 有明确消费进度；“消息很多”或“传得很快”只是吞吐层面的指标。没有 append-only log，系统缺少审计和重建能力；没有 offset tracking，processor 一重启就容易漏事件或重复做副作用。

这也解释了为什么本视频反复强调事件溯源。agent 的输入、模型响应、工具调用、错误、UI 投影和安全提示一旦都写成事件，durable stream 就成了系统的时间轴。时间轴可重放，状态就可解释；进度可跟踪，分布式 processor 就能独立部署、离线、恢复和追赶。

8.5 Kafka 与 PubSub: offset 谁来跟踪

讲者在 01:01:43–01:02:07 对比了两类消费进度模型。第一类接近 Kafka：客户端或消费者组记录自己已经消费到的 offset。第二类接近 Google Cloud Pub/Sub、AWS 的 pub/sub 系统：服务端为订阅维护确认状态，把下一条该处理的消息交给客户端。

Kafka 风格的优点是控制力强。消费者可以决定何时提交 offset，可以为了调试回退到旧 offset，也可以把不同处理器的进度放在自己的状态存储中。它像读一本带页码的书：读者自己夹书签，想重读第 120 页可以自己翻回去。代价是客户端要更懂协议，要处理提交时机、重复消费和消费者组协调。

PubSub 服务端跟踪风格的优点是使用简单。客户端收到消息、处理完成、ack，服务端负责记住这条订阅的下一步。它像快递站按签收记录派件：用户只要确认收货，站点负责安排下一件。代价是回放、精细控制、多消费者语义和审计通常更依赖平台提供的订阅能力。

维度	Kafka 风格	PubSub 风格
offset 归属	客户端或消费者组提交 consumed offset	服务端按 subscription 跟踪确认状态
重放控制	客户端更容易主动 seek 到旧 offset	依赖服务端保留期、快照或重放能力
使用复杂度	客户端责任更重	接入更省心
适合场景	需要精细恢复、审计、批处理、回放的 processor	需要托管投递、简单 ack、弹性消费的服务

对本视频的 agent 架构来说，两种都可行。关键不在于必须复制 Kafka 或某个云厂商，而在于系统必须回答同一个问题：每个 processor 到底处理到哪个 offset？没有这个答案，RAG processor、安全 checker、UI projection 和动态 worker 的协作都会变成猜测。

8.6 public namespace 的认证、授权与 client provenance

最后一段讨论转向 public namespace。讲者说，公开 namespace 很有吸引力：任何人都能复制一段 `curl`，向某条 stream 追加事件，立刻得到一种 agent 体验。这对 workshop、demo、临时协作很有价值。问题也很直白：公开写入面一旦接入真实 agent，就必须有认证、授权、来源证明和清理策略。

Authentication（认证） 回答“你是谁”。常见做法包括 API key、OAuth token、signed request、session cookie、mTLS 等。**Authorization（授权）** 回答“你能做什么”。同一个用户可能读某条 stream，却不能写；能追加普通输入事件，却不能追加 `make.stream.public` 或 `tool.result`；能触发低风险工具，却不能触发转账、发邮件、删资源等高风险动作。

Client provenance（客户端来源证明） 是附在事件上的来源、权限和可信度元数据。它至少应说明：事件来自哪个客户端、认证方式是什么、该客户端当时被授予哪些能力、服务端对这个身份判断有多大把握、请求经过了哪些代理或签名链路。可以把它写成事件信封的一部分：

$$e_i = (\tau_i, p_i, o_i, c_i, \rho_i),$$

其中 ρ_i 表示 provenance。它像海关单据上的发货方、承运方、签章和检查记录；payload 说明箱子里是什么，provenance 说明这箱东西从哪来、凭什么能进来、出了事该追到谁。

public namespace 不能放 secrets。OpenAI API key、数据库密码、内部 webhook secret、用户私有 token 都不应出现在公开事件 payload 中。公开 namespace 适合放可清理、可审计、低敏感度的实验事件；真正的 secret 应由平台侧环境变量、密钥管理服务或受控 worker 注入。

清理策略同样是安全边界的一部分。讲者明确提到，一个临时 public namespace 可以每小时清空，且公开写入面必须配合认证、授权和 client provenance。若把这句话展开成生产设计，通常还会继续补上 TTL、配额、速率限制、schema 限制和隔离策略：TTL 决定事件保留多久；配额限制单个用户或来源能写多少；速率限制防止刷爆 processor；schema 限制降低伪造高权限事件的空间；隔离策略确保 demo namespace 的事件不会污染生产 namespace。

更进一步，权限变更本身也可以事件化。比如在 stream 中追加 `make.stream.public`，此后某些客户端获得写入能力；追加 `revoke.client`，某个 key 失效；追加 `policy.updated`，processor 读到新策略后改变行为。这样做的好处是权限历史可审计，但底层认证校验仍需由服务端强制执行。事件可以描述策略变化，不能替代入口处的访问控制。

8.7 全文收束：一个事件流，多个 processor，可审计副作用

回看整场分享，讲者真正推动的是一种 agent harness 的工程分解：把 agent 运行中发生的事写进 event stream，把状态计算放进 reducer，把外部动作放进 after-append，把 UI、RAG、安全、审计和动态执行拆成独立 processor。这样，系统获得了几个很实际的能力：可重放、可调试、可扩展、可远程接入、可用 offset 恢复。

这套架构的锋利处也在这里。它允许第三方 prompt-injection checker 运行在另一台机器，凭

事件流在 LLM request 前短暂介入；它允许 RAG processor 尽力补上下文，没赶上也不拖垮主流程；它允许动态 worker 和外部服务围绕同一条 durable stream 协作；它也迫使系统认真面对认证、授权、provenance、secret 隔离和清理策略。

最终的设计取舍可以压缩成一句工程原则：主路径要短，事实流要全，外部 processor 要有机会介入，权限边界要由服务端硬性守住。事件流让 agent 的每一步都有坐标，offset 让每个 processor 知道自己走到哪里，provenance 让每个事件能追溯来源，eventual consistency 让分布式插件在有限等待内协作。少了其中任何一块，系统都会从可审计的 agent harness 退化成一串难以解释的回调和日志。

8.8 本章小结

本章完成全片收束。第三方 prompt-injection/safety checker 可以部署在另一台机器上，通过事件流观察 agent 即将发起的 LLM request，并在 $w = 200\text{ms}$ 左右的窗口内追加安全上下文； $P_j(E) \rightarrow E'$ 表示第 j 个 processor 观察既有事件流并通过追加事件形成新事件序列。讲者反对第三方 before hook，理由很直接：它会把外部服务放进主路径，放大性能退化、成本上升和 context caching 失效。

Eventual consistency 在这里表示“赶上就参与本轮，赶不上主流程继续，后续仍可通过事件收敛”。RAG processor 可以及时补充上下文，也可以晚到后供下一轮、审计或修正使用。Durable stream 则是 append-only event log 加 offset tracking；Kafka 风格常由客户端或消费者组跟踪 consumed offset，PubSub 风格常由服务端按 subscription 跟踪确认状态。最后，public namespace 需要认证、授权、client provenance、TTL/配额/速率限制/清理策略，并且绝不能承载 secrets。全片的落点是：用一条持久事件流连接多个 processor，让 agent 的状态、上下文、安全检查和副作用都能被追踪、恢复和解释。